MANUAL DEL PROYECTO: GALERÍA DE ARTE PROPIA

Asignatura: Proyecto Intermodular

Alumno: Diego Guerrero Ramos

2ºDAW A

Contenido

1. Introducción y Justificación del Proyecto 3
2. PRIMER TRIMESTRE: Concepción, Toma de Requisitos y Diseño Arquitectónico 11
12-septiembre-2025 / Presentación de Asignatura y Proyecto 11
19-septiembre-2025 / Creación de Imagen Corporativa de la Empresa 15
26-septiembre-2025 / Elaboración de Contrato y Recogida de Necesidades 19
3-octubre-2025 / Definición de Requisitos Funcionales y No Funcionales 23
10-octubre-2025 / Desarrollo de las Interfaces Gráficas (UI/UX) 30
17-octubre-2025 / Desarrollo de la Estructura de la Base de Datos 35
24-octubre-2025 / Definición de Modelo Relacional de la Base de Datos 40
31-octubre-2025 / Presentación a la Empresa de Interfaces y Base de Datos 47
7-noviembre-2025 / Elección de Tecnologías Para Utilizar 50
14-noviembre-2025 / Estructuración Inicial de Documentación y Arquitectura Base 56
21-noviembre-2025 / Definición de Puntos de los Manuales de Usuario y Técnico 61
3. SEGUNDO TRIMESTRE: Desarrollo Documental, Pruebas y Despliegue 64
05-diciembre-2025 / Desarrollo Inicial de Manuales de Usuario y Técnico (Fase de Abstracción y Documentación) 64
12-diciembre-2025 / Análisis Arquitectónico y Opciones de Despliegue de Aplicativos 68
19-diciembre-2025 / Pruebas de Despliegue en Entorno Local (TomCat / Servidor HTTP Local) 71
09-enero-2026 / Pruebas de Despliegue en Vercel (Producción) 76
16-enero-2026 / Fase de Pruebas Extensivas (QA Testing) y Corrección de Errores (Bugs) 80
23-enero-2026 / Revisión Final y Auditoría de Documentación 86
30-enero-2026 / Preparación para la Presentación y Defensa 89
06-febrero-2026 / Implementación de Feedback y Accesibilidad Web (WCAG) 93
13-febrero-2026 / Optimización de Base de Datos y Revisión de Ciberseguridad 97
20-febrero-2026 / Generación de Datos Simulados (Mock Data) y Pruebas de Estrés 102
27-febrero-2026 / Cierre de Documentación y Manual de Despliegue 105
05-marzo-2026 / Congelación de Código (Code Freeze) y Versionado Semántico 109
12-marzo-2026 / Preparación de la Defensa y Creación de Material Audiovisual 111
19-marzo-2026 / Entrega Definitiva y Defensa del Proyecto 115
4. Conclusiones 118

1. Introducción y Justificación del Proyecto

El presente documento constituye la memoria técnica, funcional y descriptiva, así como el registro cronológico exhaustivo, del trabajo de ingeniería de software y diseño web realizado para el desarrollo del aplicativo "Galería de Arte Propia". Este proyecto integral se ha concebido desde cero para dar respuesta a la necesidad inminente de una artista contemporánea de digitalizar su modelo de negocio, centralizar su presencia en internet y establecer un canal de venta directo (Direct-to-Consumer o D2C) con sus coleccionistas y seguidores a nivel global.

Históricamente, el mercado del arte ha estado fuertemente intermediado. Las galerías físicas y los marchantes de arte tradicionales suelen aplicar comisiones que oscilan entre el 40% y el 50% sobre el valor final de la obra, además de imponer restricciones creativas, calendarios de exhibición muy rígidos y condiciones de exclusividad que limitan el crecimiento orgánico del creador. Este proyecto nace de la necesidad real y urgente de democratizar y descentralizar este proceso transaccional y expositivo. El objetivo es otorgar a la creadora el control absoluto sobre su inventario, la fijación de sus precios, la narrativa de su imagen de marca y, lo más importante, la propiedad de los datos y la relación directa y sin filtros con su audiencia compradora.

Para lograr este objetivo, se ha diseñado una solución tecnológica híbrida que fusiona las capacidades transaccionales de un E-commerce de alto rendimiento con las mecánicas de retención de una plataforma de comunidad exclusiva (Membership site). El sistema no se limita a ser un mero catálogo digital o un portafolio estático tradicional; es una herramienta viva, asíncrona y plenamente operativa que permite transaccionar económicamente a través de una pasarela de pago real y segura, garantizando la inmediatez que exige el consumidor digital actual.

Uno de los pilares fundamentales que justifican esta arquitectura de software es la segregación de la experiencia de usuario (UX) en dos vertientes claramente diferenciadas, lo que permite crear un embudo de conversión (funnel) mucho más efectivo:

- El usuario anónimo (Público general): Tiene acceso a un escaparate visual inmersivo, donde puede explorar las colecciones abiertas, conocer la biografía e historia de la artista y realizar compras rápidas mediante un flujo de "Checkout" optimizado para invitados. Esta ruta busca reducir la fricción al mínimo, permitiendo la compra por impulso sin la barrera de obligar al usuario a rellenar largos formularios de registro.

- El usuario registrado (Coleccionista VIP): Constituye el núcleo de la estrategia de fidelización. Al ceder sus datos y crear una cuenta verificada, el sistema recompensa al usuario desbloqueando una "landing page" privada y un panel de control personalizado. Este entorno actúa como un

club exclusivo donde la artista publica noticias restringidas, diarios de su proceso creativo, vídeos en el estudio y, como principal atractivo comercial, ofrece acceso anticipado (preventas) a futuras exposiciones o nuevas obras originales antes de que su disponibilidad se abra al público general.

La metodología de trabajo implementada a lo largo del ciclo de vida del desarrollo ha seguido un enfoque "User-Centric" (Centrado en el Usuario) y "Design-First" (Diseño guiado por la estética y funcionalidad). Esto garantiza que la compleja lógica subyacente de bases de datos relacionales, manejo de sesiones y enrutamiento quede completamente oculta tras una interfaz limpia, minimalista e intuitiva, inspirada en el concepto del "Cubo Blanco" de las galerías físicas, donde el protagonismo visual recae enteramente en la alta resolución de las obras de arte.

A continuación, se detalla la base estratégica del proyecto mediante el análisis del entorno y de los actores involucrados, paso previo y fundamental antes de iniciar la arquitectura del software.

Análisis de Interesados (Stakeholders)

Para garantizar el éxito de la plataforma "Galería de Arte Propia" y definir correctamente las fronteras de acceso, los roles y los permisos del sistema informático, se realizó una exhaustiva identificación de los interesados. Esto permitió comprender las expectativas y el nivel de influencia de cada grupo en la toma de decisiones arquitectónicas y de diseño.

- Artista / Creadora:

  * Rol en el Proyecto: Cliente y Administradora (Superusuario).

  * Nivel de Influencia: Alta.

  * Nivel de Interés: Alto.

  * Expectativas y Requisitos Principales: Disponer de un panel de control (CMS) privado e intuitivo. Necesita capacidad total para realizar operaciones CRUD (Crear, Leer, Actualizar, Borrar) sobre sus colecciones, fijar y modificar precios, editar descripciones técnicas de las obras y publicar noticias exclusivas. Exige la recepción de pagos de forma segura e instantánea, además de un control total y automatizado de su inventario para evitar sobreventas de piezas únicas.

- Coleccionistas / Seguidores Fieles:

  * Rol en el Proyecto: Usuario Registrado (Actor VIP).

  * Nivel de Influencia: Media.

  * Nivel de Interés: Alto.

  * Expectativas y Requisitos Principales: Exigen una plataforma estable y libre de errores. Su motivación principal es el acceso a una zona privada con contenido de alto valor (noticias, making of, inspiración). Requieren la posibilidad de comprar obras en primicia para asegurarse piezas muy demandadas. Esperan una pasarela de pago fluida, transparente y la capacidad de consultar el seguimiento histórico y el estado de sus pedidos desde su perfil.

- Público General:

  * Rol en el Proyecto: Usuario Anónimo (Invitado).

  * Nivel de Influencia: Baja.

  * Nivel de Interés: Medio.

  * Expectativas y Requisitos Principales: Buscan una navegación extremadamente rápida e intuitiva desde dispositivos móviles. Esperan un catálogo visualmente atractivo que cargue las imágenes sin demora. Su requisito principal es un proceso de compra sin fricciones que no les obligue a crear una cuenta si desean realizar una adquisición puntual (Guest Checkout).

- Proveedor de Pagos (Stripe):

  * Rol en el Proyecto: Infraestructura / Actor Externo del Sistema.

  * Nivel de Influencia: Alta.

  * Nivel de Interés: Bajo (proveedor automatizado).

  * Expectativas y Requisitos Principales: Impone que la aplicación implemente correctamente sus APIs REST y la escucha de Webhooks asíncronos. Exige el cumplimiento estricto con las normativas internacionales de seguridad (como la certificación PCI-DSS) para la tokenización y el manejo cifrado de tarjetas de crédito.

- Desarrollador Web:

  * Rol en el Proyecto: Creador, Arquitecto e Integrador (Alumno).

* Nivel de Influencia: Alta.

* Nivel de Interés: Alto.

* Expectativas y Requisitos Principales: Diseñar y codificar una arquitectura moderna y escalable utilizando React JS y Supabase. Desplegar el aplicativo en un entorno robusto (Vercel) garantizando un alto rendimiento. Cumplir de manera sobresaliente con todos los requisitos académicos, técnicos y de documentación exigidos para la evaluación y viabilidad del Trabajo Final.

Análisis DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades)

Para evaluar la viabilidad comercial y técnica de la plataforma digital frente al ecosistema actual de venta de arte, se procedió a realizar un análisis estratégico DAFO. Este análisis cruzado justifica de manera analítica la elección del stack tecnológico y el enfoque de negocio adoptado.

ANÁLISIS INTERNO:

- FORTALEZAS (F):

* Máxima Rentabilidad: Se obtiene un margen de beneficio del 100% para la artista al eliminar por completo las altas comisiones y recargos exigidos por las galerías físicas y plataformas de intermediarios de terceros.

* Vanguardia Tecnológica: El uso de tecnologías modernas, asíncronas y altamente escalables (React JS en el Frontend y arquitectura Serverless con Supabase en el Backend) garantiza tiempos de carga mínimos, cruciales para retener al usuario.

* Retención y Comunidad: La creación de un canal de fidelización directo mediante el sistema de usuarios registrados y noticias exclusivas genera un fuerte sentimiento de pertenencia y aumenta el Valor del Tiempo de Vida del Cliente (LTV).

* Eficiencia de Costes: El coste de infraestructura tecnológica y de mantenimiento mensual es prácticamente nulo gracias a la optimización de los planes gratuitos o de entrada de servicios en la nube como Vercel y Supabase.

- DEBILIDADES (D):

* Complejidad Logística Manual: Aunque la plataforma automatiza la venta y el cobro, la artista debe asumir manualmente la carga de trabajo físico que supone el embalaje seguro, la gestión de seguros y la cotización aduanera de los envíos internacionales de obras físicas.

* Dependencia Transaccional: Existe una dependencia de una única pasarela de pagos (Stripe). Si la cuenta sufre una retención de fondos o un bloqueo temporal por algoritmos antifraude (falsos positivos), las ventas online se paralizan momentáneamente.

* Curva de Aprendizaje Operativa: Requiere un esfuerzo inicial de adaptación para que la artista (sin perfil técnico) se acostumbre a

autogestionar su inventario digital, subir imágenes optimizadas y redactar artículos a través del panel de administración construido.

ANÁLISIS EXTERNO:

- OPORTUNIDADES (O):

* Expansión Global Inmediata: El mercado digital carece de fronteras. La plataforma ofrece la posibilidad de vender, cobrar en múltiples divisas y enviar obras a coleccionistas internacionales operando las 24 horas del día, los 365 días del año.

* Crecimiento del E-commerce Premium: Existe una tendencia consolidada de crecimiento del comercio electrónico en el sector del lujo y el arte, con usuarios cada vez más confiados y habituados a realizar compras de importes elevados a través de internet.

* Tráfico Orgánico y Social: La posibilidad de utilizar las redes sociales ya consolidadas de la artista (Instagram, TikTok, Pinterest) como un potente embudo de conversión directo (Funnel) hacia la nueva plataforma web, redirigiendo a su audiencia cautiva hacia su propio canal de venta.

- AMENAZAS (A):

* Saturación del Mercado: El ecosistema digital está altamente saturado de galerías online, plataformas masivas y otros artistas independientes, lo que exige que la calidad del producto web sea excepcional a nivel visual y de rendimiento para lograr destacar.

* Rigurosidad Normativa: Existen exigencias legales y de ciberseguridad muy estrictas, especialmente el Reglamento General de Protección de Datos (RGPD) en Europa, que imponen fuertes multas si no se gestionan correctamente las bases de datos de correos electrónicos y el consentimiento de cookies.

* Dependencia de Infraestructuras de Terceros: Existen riesgos inherentes a posibles caídas temporales de los servicios en la nube proveedores (como Vercel, Supabase o GitHub), aunque este riesgo está fuertemente mitigado por los Acuerdos de Nivel de Servicio (SLA) del 99.9% de Uptime que ofrecen estas grandes corporaciones tecnológicas.

2. PRIMER TRIMESTRE: Concepción, Toma de Requisitos y Diseño Arquitectónico.

12-septiembre-2025 / Presentación de Asignatura y Proyecto

- Contexto Académico y Planteamiento Inicial de Objetivos:

La sesión inaugural del curso sirvió para establecer de manera formal las normativas académicas, los plazos de entrega, los hitos de evaluación y los criterios de calificación del Proyecto Intermodular (Trabajo Final). El objetivo primordial de esta primera semana de inmersión fue definir, acotar y validar la idea central del proyecto. Para ello, se aplicaron técnicas ágiles de ideación y metodologías iterativas basadas en el marco de trabajo "Design Thinking" (Pensamiento de Diseño).

Durante estas primeras sesiones de lluvia de ideas (Brainstorming), el enfoque principal fue empatizar con una problemática real, tangible y medible en el mercado actual: la barrera tecnológica y comercial a la que se enfrentan los creadores independientes. En concreto, se analizó la dificultad de los artistas plásticos para digitalizar su modelo de negocio y monetizar su obra de forma directa (modelo Direct-to-Consumer), sin depender del monopolio histórico de las galerías físicas o de plataformas intermediarias que exigen comisiones abusivas.

La meta de esta fase embrionaria fue delimitar el alcance del software (Project Scope) para asegurar que el desarrollo final fuera técnicamente abarcable en el tiempo estipulado para el curso, pero lo suficientemente complejo, escalable y robusto como para demostrar un dominio absoluto de las arquitecturas web modernas (Frontend, Backend, bases de datos relacionales y despliegue en la nube).

- Desarrollo Técnico e Investigación del "Estado del Arte":

Una vez definida la premisa inicial, se procedió a realizar un estudio técnico y comercial exhaustivo del "Estado del Arte" en el sector del mercado del arte digital (ecosistema ArtTech) y del comercio electrónico especializado en artículos de lujo o alto valor (High-Ticket E-commerce).

Se auditaron y diseccionaron en profundidad las plataformas líderes en la intermediación de obras de arte, tales como Artsy, Saatchi Art o Singulart,

así como soluciones de software como servicio (SaaS) generalistas orientadas a la creación web, como Shopify, Wix o Squarespace. Durante esta auditoría competitiva y análisis heurístico, se detectaron varias deficiencias críticas en la forma en que los artistas independientes gestionan su presencia web en la actualidad:

1. Portafolios Estáticos y Carentes de Lógica de Negocio: La inmensa mayoría de las páginas web de artistas operan como meros portafolios de exhibición visual (Lookbooks). Carecen de bases de datos relacionales por detrás, de gestión dinámica de inventario y, lo más crítico, de pasarelas de pago integradas. Son webs de "solo lectura" que no permiten la transacción económica.

2. Fricción Transaccional y el Síndrome del "Precio Bajo Demanda": Se identificó un patrón muy negativo en el sector: si un coleccionista desea adquirir una obra, generalmente se ve obligado a rellenar un formulario de contacto genérico o enviar un correo electrónico manual para preguntar por la disponibilidad, los costes de envío y el precio final de la pieza. Este proceso analógico, opaco y asíncrono genera una latencia inaceptable en el comercio digital moderno. Psicológicamente, la barrera de tener que "pedir permiso o información" para comprar resulta en una tasa de abandono de la intención de compra (Bounce Rate) superior al 80%.

3. Pérdida del Activo de Datos y Falta de Fidelización: Cuando un artista delega sus ventas a plataformas de terceros (como las mencionadas Saatchi

o galerías físicas), el intermediario retiene la propiedad de los datos de los compradores (correos electrónicos, perfiles demográficos, historiales de compra y preferencias estéticas). Esto impide sistemáticamente que el creador construya una base de datos propia (First-Party Data) para nutrir a su comunidad, realizar campañas de email marketing o lanzar preventas exclusivas (Newsletters). El artista pierde el control sobre el ciclo de vida de su propio cliente.

- Metodología de Validación Aplicada:

Para validar que la solución de software propuesta iba a resolver estos problemas, se aplicaron principios de la metodología "Lean Startup". Se definió cuál sería el Producto Mínimo Viable (MVP) necesario para salir al mercado: una plataforma que no solo muestre arte, sino que permita procesar un pago de manera instantánea y recoja el correo del comprador de forma automatizada. Se determinó que cualquier funcionalidad que no aportara valor directo a la venta o a la fidelización sería descartada de la primera fase de desarrollo.

- Conclusiones y Pivote Estratégico del Proyecto:

Tras analizar los datos empíricos recabados en la investigación de mercado, se tomó la decisión estratégica de pivotar la idea original del proyecto académico. Se descartó por completo la programación de un simple catálogo web corporativo y se estableció como directriz principal el desarrollo de un sistema transaccional robusto, asíncrono y de ciclo completo (End-to-End).

Bajo esta nueva premisa, la aplicación "Galería de Arte Propia" se conceptualizó oficialmente como un E-commerce híbrido que integraría una funcionalidad avanzada de "Membership" (Comunidad privada). El nuevo enfoque tecnológico buscaría la "cero fricción" en el proceso de pago mediante la futura integración de la API de Stripe, y apostaría por un modelo de captación de *leads* (contactos) mediante el cual los usuarios, a cambio de registrarse en la plataforma, obtendrían beneficios exclusivos. Este cambio de paradigma sentó las bases inamovibles para todos los requisitos técnicos de modelado de bases de datos, autenticación de usuarios y enrutamiento dinámico que se definirían en las semanas posteriores del trimestre.

19-septiembre-2025 / Creación de Imagen Corporativa de la Empresa

- Contexto y Objetivos Generales del Diseño Visual:

Enmarcado en la segunda semana de desarrollo, y antes de proceder a la escritura de la primera línea de código funcional o estructuración de bases de datos, era un requisito imperativo definir la identidad visual y el "Design System" (Sistema de Diseño) de la plataforma. En el sector del "High-Ticket E-commerce" (comercio electrónico de alto valor) y específicamente en el mercado del arte contemporáneo, la confianza del consumidor está intrínsecamente ligada a la percepción de calidad, exclusividad y profesionalidad que transmite la interfaz de usuario (UI).

El objetivo principal de esta fase fue traducir la esencia artística de la creadora a un entorno digital. Se tomó como referencia conceptual el estándar museístico e internacional del "Cubo Blanco" (White Cube). Esta teoría expositiva dicta que el entorno arquitectónico (o en este caso, la arquitectura de la interfaz web) debe ser completamente neutro, aséptico y carente de cualquier estímulo visual que compita con la obra expuesta. La interfaz web debe ser "invisible"; un contenedor silencioso cuyo único propósito es elevar y ceder todo el protagonismo cromático y emocional a las fotografías de alta resolución de las obras de arte.

- Desarrollo Técnico: Psicología del Color y Paleta Cromática:

Para lograr este efecto de "invisibilidad" y sofisticación, se descartó el uso de paletas de colores vibrantes o gradientes complejos en la estructura de la aplicación. Se definió una paleta monocromática estricta y altamente contrastada, fundamentada en los siguientes tokens de diseño:

1. Fondos Base (Backgrounds): Se seleccionó un blanco roto o "Off-White" (Código HEX: #FAFAFA o #F8FAFC) en lugar de un blanco puro (#FFFFFF). El blanco puro en pantallas digitales (especialmente en monitores OLED o pantallas de dispositivos móviles con brillo alto) genera fatiga visual severa al rebotar demasiada luz directa a los ojos del usuario. El uso de tonos alabastro o gris perla muy sutiles permite sesiones de navegación mucho más largas y cómodas, invitando al coleccionista a explorar el catálogo sin cansancio.

2. Textos Principales y Jerarquía de Lectura: Para los cuerpos de texto, descripciones de obras y biografías, se huyó del negro absoluto (#000000), optando por un gris carbón profundo (Código HEX: #171717 o text-neutral-900 en la escala de Tailwind CSS). Esto suaviza el contraste y mejora notablemente la legibilidad en bloques de texto largos, cumpliendo con los estándares de accesibilidad web (WCAG) de nivel AA.

3. Llamadas a la Acción (Call to Action / CTA): El color de acento, reservado única y exclusivamente para los botones transaccionales críticos (como "Añadir al Carrito", "Proceder al Pago" o "Registrarse en la Comunidad VIP"), se definió en negro puro (#000000) con texto en blanco. Esta decisión estratégica busca guiar el ojo del usuario de forma inequívoca hacia la conversión, eliminando cualquier ambigüedad sobre dónde debe hacer clic para adquirir una obra.

- Selección Tipográfica (Typography) y Escala Modular:

La tipografía es, junto con el espacio en blanco (White Space), el elemento de diseño más importante en una galería digital. Se estableció una combinación tipográfica (Font Pairing) dual para separar claramente la narrativa artística de la información técnica y transaccional:

1. Tipografía Primaria (Títulos y Encabezados): Se integró una fuente de la familia Serif (con remates), concretamente "Playfair Display". Las fuentes Serif transmiten históricamente herencia, lujo, autoridad editorial y elegancia clásica. Esta tipografía se utiliza exclusivamente para los nombres

de las colecciones, los títulos de las obras (H1, H2) y las cabeceras de las noticias exclusivas para los usuarios registrados.

2. Tipografía Secundaria (Interfaz, Precios y Datos Técnicos): Para equilibrar el clasicismo de la Serif y aportar un toque de modernidad, se seleccionó la fuente "Inter", una Sans-Serif (sin remates) diseñada específicamente para interfaces de usuario en pantallas digitales. Se utiliza para los párrafos, la navegación del menú, las etiquetas de estado ("Vendida", "Disponible"), las medidas físicas de los cuadros y, de manera crítica, para los precios. Los números en "Inter" tienen un ancho tabular (Tabular Numerals), lo que significa que todos los dígitos ocupan el mismo espacio horizontal, permitiendo que los precios se alineen perfectamente en el carrito de compras y en las facturas generadas.

- Iconografía y Uso del Espacio Negativo (White Space):

Se decidió utilizar la librería de iconos "Lucide React" por su peso ligero en kilobytes y su diseño minimalista basado en trazos (Stroke-based icons) de 1.5px de grosor. Se incorporaron iconos muy sutiles para el carrito de compras, el perfil de usuario y los menús desplegables. Además, se implementó un uso agresivo del espacio negativo (márgenes amplios o "Paddings" de escala p-8 o p-12). En diseño web orientado al lujo, el espacio vacío no es un espacio desperdiciado; es el elemento que confiere "aire", magnitud y valor a la obra expuesta.

- Desafíos Técnicos y Solución en Código (Tailwind CSS Configuration):

El mayor desafío organizativo en esta etapa fue asegurar que estas decisiones de diseño no se perdieran o se aplicaran de forma inconsistente por error humano durante los meses posteriores de codificación.

Solución: En lugar de "quemar" (hardcodear) estos códigos de color y fuentes en los archivos individuales, se procedió a parametrizar todo el Sistema de Diseño inyectándolo directamente en el archivo de configuración maestro de nuestro framework de estilos (`tailwind.config.js`). Se extendió el tema (theme extension) por defecto de Tailwind para incluir nuestras variables personalizadas (`fontFamily: { serif: ['Playfair Display', 'serif'], sans: ['Inter', 'sans-serif'] }` y `colors: { gallery: { black: '#171717', white: '#FAFAFA' } }`).

Esta decisión arquitectónica de Frontend garantiza que toda la aplicación mantenga una cohesión visual absoluta. Si en el futuro la artista decide cambiar la tipografía de toda su marca, bastará con modificar una sola línea de código en este archivo de configuración, y el cambio se propagará instantáneamente a todas las páginas de la galería virtual, reduciendo el coste de mantenimiento a cero.

26-septiembre-2025 / Elaboración de Contrato y Recogida de Necesidades

- Contexto, Consultoría y Objetivos Fundamentales:

Una vez definida la identidad visual y el marco estético de la plataforma, la tercera semana del proyecto se dedicó íntegramente a la fase de

consultoría tecnológica, levantamiento de requisitos y formalización del alcance del proyecto (Project Scope). Este hito es crítico en cualquier ciclo de vida de desarrollo de software (SDLC), ya que marca la transición de las ideas abstractas a las especificaciones técnicas concretas.

El objetivo principal fue realizar una simulación de reuniones de "Discovery" (Descubrimiento) con la creadora/artista, actuando ella como el "Stakeholder" principal o "Product Owner". Durante estas sesiones, se redactó el Documento de Alcance y el SLA (Service Level Agreement o Acuerdo de Nivel de Servicio). Este documento contractual (a nivel académico) sirvió para blindar los límites del desarrollo, estableciendo claramente qué funcionalidades entrarían en el Producto Mínimo Viable (MVP) para la defensa del proyecto y cuáles quedarían relegadas a futuras versiones (v2.0), evitando así el temido "Scope Creep" (corrupción o expansión descontrolada del alcance).

- Metodología de Recogida de Necesidades:

Para extraer los requisitos de negocio, se utilizaron técnicas de entrevistas estructuradas y la elaboración de "Customer Journeys" (Mapas de Viaje del Cliente). Se analizó paso a paso qué acciones realizaría un usuario desde que aterriza en la página web a través de un enlace de Instagram, hasta que completa el pago con su tarjeta de crédito.

Durante este proceso de ingeniería de requisitos, emergió una necesidad legal y logística fundamental: al tratarse de obras de arte físicas (bienes

tangibles) y no de descargas digitales, la plataforma debía contemplar de manera obligatoria la recogida estructurada de direcciones de envío internacionales, el cálculo de variables de disponibilidad (Stock = 1 para piezas originales) y la aceptación explícita de los Términos y Condiciones de Venta (T&C) y Políticas de Devolución, cumpliendo así con el marco normativo del comercio electrónico europeo.

- Desarrollo Técnico y Estructuración de Actores del Sistema:

Fruto de esta recogida de necesidades, se determinó que la arquitectura de la aplicación debía soportar tres perfiles de usuario radicalmente distintos, cada uno con un embudo de conversión y un nivel de autorización criptográfica diferente. Se procedió a modelar el sistema en base a los siguientes "Actores":

1. Actor 1: El Administrador (La Artista / Backoffice).

- Perfil con el máximo nivel de privilegios. Requiere un panel de control privado (Dashboard o CMS propio) oculto al público.

- Necesidades Críticas: Capacidad para realizar un CRUD (Crear, Leer, Actualizar, Borrar) completo de su catálogo. Necesita una interfaz sencilla para subir fotografías de alta resolución de las obras, asignarles metadatos (título, año, técnica, dimensiones en centímetros) y vincularlas a una "Colección" específica.

- Necesidades de Gestión: Un módulo para redactar y publicar las "Noticias" (Blog), pudiendo marcar con un interruptor (Toggle) si dicha noticia es pública o si está restringida solo para la comunidad VIP.

- Necesidades Financieras: Un módulo de "Pedidos" donde pueda visualizar qué obras han sido pagadas, los datos del comprador y el estado del envío para poder gestionar la logística física desde su estudio.

2. Actor 2: El Usuario Registrado (Coleccionista VIP / Frontoffice Privado).

- Perfil de usuario autenticado mediante correo electrónico y contraseña (gestionado de forma segura por Supabase Auth).

- Necesidades Críticas: Su motivación principal es la exclusividad. Requiere acceso a rutas protegidas (Protected Routes en React) a las que un usuario normal no puede llegar. En estas rutas privadas consumirá el contenido de valor añadido: noticias sobre el proceso creativo, diarios del artista e invitaciones a preventas.

- Necesidades Transaccionales: Un panel de usuario donde pueda gestionar sus direcciones de envío guardadas para acelerar futuras compras (reduciendo la fricción del Checkout) y un historial detallado de las obras que ha adquirido a lo largo del tiempo, con la posibilidad de descargar sus facturas o recibos en formato PDF.

3. Actor 3: El Usuario Anónimo (Público General / Frontoffice Público).

- Perfil efímero y sin estado persistente (hasta que añade algo al carrito).

- Necesidades Críticas: Una navegación fluida por la galería abierta. Su única necesidad es un canal visual libre de barreras para admirar el arte y leer la biografía de la autora.

- Necesidades Transaccionales: Si decide comprar una obra de acceso público, el sistema debe permitirle realizar un "Guest Checkout" (Compra como invitado). Se estipuló como requisito innegociable no obligar a este usuario a crear una cuenta para finalizar una transacción, maximizando así la tasa de conversión en compras impulsivas.

- Conclusiones y Definición del Enrutamiento Frontend:

La principal conclusión técnica extraída de esta semana de consultoría fue la necesidad de construir tres "Frontends" conceptuales que convivirían bajo la misma Single Page Application (SPA) de React.

A nivel de ingeniería, esto implicó la necesidad de planificar un sistema de enrutamiento dinámico y protección de rutas (Route Guards). El código fuente debería ser capaz de verificar asíncronamente en cuestión de milisegundos el estado de autenticación (Token JWT) del usuario que intenta acceder a una URL. Si un usuario anónimo intenta forzar la entrada a la ruta "/vip/preventas", el sistema debe interceptar la petición, denegar el acceso y redirigirlo (Redirect) automáticamente a la página de inicio de sesión (Login), garantizando así la exclusividad y seguridad de los datos de la plataforma.

3-octubre-2025 / Definición de Requisitos Funcionales y No Funcionales

- Contexto y Objetivos del Documento SRS:

Tras finalizar la fase de descubrimiento y consultoría, la cuarta semana del proyecto se dedicó a la elaboración del documento SRS (Software Requirements Specification o Especificación de Requisitos de Software). El objetivo primordial de esta fase es la traducción literal de las necesidades de negocio e intenciones comerciales de la artista a especificaciones técnicas concretas, medibles y programables por un ingeniero de software.

En un desarrollo ágil, no basta con decir "la web debe vender cuadros". Es necesario desglosar cada interacción en unidades atómicas de información que el equipo de desarrollo pueda estimar en tiempo y esfuerzo. Para estructurar este volumen de peticiones y priorizar el desarrollo del Producto Mínimo Viable (MVP), se aplicó la metodología de priorización MoSCoW, descartando el formato tabular tradicional en favor de un desglose analítico en formato de lista para mayor profundidad descriptiva.

- Requisitos Funcionales (Metodología MoSCoW):

* M - MUST Have (Debe tener obligatoriamente):

Son aquellas funcionalidades críticas y troncales sin las cuales la aplicación carece de sentido, utilidad comercial o viabilidad legal. Representan el núcleo duro del MVP para la entrega del proyecto.

- Catálogo Dinámico de Obras: El sistema debe extraer de la base de datos (Supabase) un listado de obras de arte, mostrando para cada una su imagen principal, título, técnica, dimensiones físicas, año de creación y precio de venta al público.

- Pasarela de Pago Integrada (Checkout): El sistema debe permitir el procesamiento de tarjetas de crédito/débito en tiempo real mediante la integración de la API de Stripe, generando una sesión de pago segura (Stripe Checkout Session).

- Gestión de Inventario Estricta: Tratándose de obras de arte originales y únicas, el sistema debe bloquear la compra simultánea. Una vez el pago es confirmado (vía Webhook), el estado de la obra en la base de datos debe pasar automáticamente de "Disponible" a "Vendida" (Out of Stock), deshabilitando el botón de compra para futuros visitantes.

- Panel de Administración (CMS CRUD): La artista debe tener acceso a una ruta protegida donde pueda Crear, Leer, Actualizar y Borrar obras, colecciones y noticias, mediante formularios validados.

- Autenticación Segura: Implementación de un sistema de registro y login (Sign Up / Sign In) mediante correo electrónico y contraseña cifrada para la creación de la comunidad VIP de coleccionistas.

* S - SHOULD Have (Debería tener):

Funcionalidades de altísimo valor añadido que mejoran drásticamente la experiencia de usuario (UX) y la conversión, pero cuya ausencia no impide el flujo básico de compra.

- Zona Privada VIP (Membership): Un "Dashboard" o área personal para el usuario registrado donde pueda consumir las noticias exclusivas y ver las obras en preventa antes que el público general.

- Historial de Pedidos: Los usuarios autenticados deberían poder consultar el estado de sus compras anteriores y descargar los recibos de las transacciones.

- Filtrado y Categorización: Un sistema asíncrono en la galería pública que permita al usuario filtrar las obras por "Colección", "Técnica" (ej. Óleo, Acrílico, Digital) o "Estado" (Disponibles vs. Vendidas) sin recargar la página.

- Visor de Imágenes Avanzado (Lightbox): Al hacer clic en la fotografía de una obra, esta debería abrirse a pantalla completa permitiendo hacer zoom para apreciar la textura de las pinceladas, un factor de decisión de compra clave en el arte.

* C - COULD Have (Podría tener):

Extras deseables o innovaciones tecnológicas que se implementarán única y exclusivamente si el tiempo de desarrollo (marzo-junio) lo permite, sirviendo como "Bonus" para la nota final del proyecto.

- Integración de Correos Transaccionales Automáticos: Envío de un email automatizado de "Gracias por tu compra" o "Bienvenido al club VIP" utilizando servicios de terceros como Resend o SendGrid.

- Modo Oscuro (Dark Mode): Un interruptor (Toggle) que permita al usuario cambiar la interfaz del "Cubo Blanco" a un "Cubo Negro" para reducir la fatiga visual en navegaciones nocturnas.

- Visualización en Realidad Aumentada (AR): Un botón que permita al usuario, desde su teléfono móvil, proyectar el cuadro en la pared de su salón usando la cámara, para comprobar cómo encajan las dimensiones.

* W - WON'T Have (No tendrá en esta versión):

Funcionalidades que se declaran explícitamente fuera del alcance del proyecto actual y se reservan para futuras iteraciones o rondas de financiación (v2.0).

- Aplicación Móvil Nativa: No se desarrollarán archivos APK para Android ni IPA para iOS. Todo el esfuerzo se centrará en una aplicación web responsiva (Web App).

- Sistema de Subastas en Tiempo Real: La complejidad de gestionar pujas en vivo mediante WebSockets excede el alcance del actual contrato de desarrollo.

- Definición de Historias de Usuario (Metodología Ágil):

Para fragmentar los macro-requisitos en tareas programables a nivel de código (Componentes de React), se redactaron Historias de Usuario bajo la sintaxis estándar de la industria del desarrollo de software: "Como [Actor], quiero [Acción] para [Objetivo]".

* Historia de Usuario 1: Compra por Impulso (Guest Checkout)

- Como: Coleccionista anónimo (Público General).

- Quiero: Poder añadir una obra al carrito y pagar introduciendo mi tarjeta y mi dirección de envío sin tener que crear una cuenta con contraseña.

- Para: Finalizar mi compra lo más rápido posible antes de arrepentirme o perder el interés, reduciendo la fricción tecnológica.

- Criterios de Aceptación: 1) El botón "Comprar ahora" debe llevar directamente a la pasarela de Stripe. 2) Tras el pago exitoso, el sistema debe redirigirme a una página de "Éxito" (Success URL) confirmando mi número de pedido.

* Historia de Usuario 2: Acceso a Contenido Restringido (Comunidad VIP)

- Como: Usuario Registrado (Seguidor Fiel).

- Quiero: Iniciar sesión en la plataforma y ver un feed de noticias que el público anónimo no puede ver.

- Para: Sentirme parte de un club exclusivo y tener ventaja a la hora de comprar nuevas colecciones antes de que se agoten.

- Criterios de Aceptación: 1) Si intento acceder a la URL "/vip/noticias" sin estar logueado, el sistema me expulsa a "/login". 2) Una vez logueado, la base de datos debe devolverme el JSON con las noticias marcadas como "privadas".

* Historia de Usuario 3: Gestión del Inventario (Artista)

- Como: Artista (Superusuario Administrador).

- Quiero: Un formulario donde pueda subir la foto de un cuadro nuevo, ponerle un título, un precio y marcarlo como "Disponible".

- Para: Actualizar mi catálogo de forma autónoma sin depender de un programador o de un webmaster que me cobre por cada actualización.

- Criterios de Aceptación: 1) El formulario debe obligarme a rellenar el precio (no puede ser nulo o negativo). 2) La foto debe subirse al "Storage" de Supabase y devolver una URL pública que se guardará en la base de datos relacional.

- Requisitos No Funcionales (Arquitectura, Seguridad y Rendimiento):

Estos requisitos no describen lo que el sistema "hace", sino "cómo" debe comportarse el sistema bajo el capó para garantizar su calidad y viabilidad a largo plazo.

* RNF1 (Seguridad y Normativa LOPD / RGPD):

Estricto cumplimiento del Reglamento General de Protección de Datos europeo. Las contraseñas de los usuarios no se guardarán en texto plano, sino que estarán "hasheadas" criptográficamente (algoritmo bcrypt) mediante los servicios de autenticación de Supabase. Asimismo, la plataforma cumplirá la normativa PCI-DSS al delegar el 100% del procesamiento y tokenización de las tarjetas de crédito a los servidores de Stripe; la base de datos de la galería jamás tocará ni almacenará un número de tarjeta (PAN) o código CVV, eximiendo a la artista de responsabilidades legales críticas frente a hackeos.

* RNF2 (Rendimiento Web y Optimización de Activos Multimedia - Core Web Vitals):

Dado que el arte es un producto puramente visual, la carga de la página (LCP - Largest Contentful Paint) debe ser inferior a 2.5 segundos incluso en redes 3G móviles. Para lograrlo, las imágenes de alta resolución de las obras se servirán mediante redes de entrega de contenido (CDN), se comprimirán a formatos de próxima generación (WebP o AVIF) y se implementará un algoritmo de "Lazy Loading" (carga diferida) en React, de forma que el navegador solo descargue las fotografías de los cuadros a medida que el usuario haga "scroll" hacia abajo por la galería, ahorrando ancho de banda masivamente.

* RNF3 (Accesibilidad Universal - WCAG 2.1):

Para garantizar la inclusión digital, todas las imágenes generadas por el sistema tendrán de forma obligatoria un atributo HTML 'alt' (texto alternativo) generado a partir de la descripción de la obra. Esto no solo es un requisito legal de accesibilidad para que los lectores de pantalla (VoiceOver, TalkBack) describan el cuadro a personas con discapacidades visuales, sino que es un pilar fundamental para el posicionamiento orgánico en buscadores (SEO) en plataformas como Google Images.

10-octubre-2025 / Desarrollo de las Interfaces Gráficas (UI/UX)

- Contexto y Objetivos del Prototipado Visual:

Una vez cerrados y aprobados los requisitos funcionales (SRS) bajo la metodología MoSCoW, el proyecto avanzó hacia la materialización visual de dichos requisitos. Esta fase, conocida como prototipado o diseño UI/UX

(User Interface / User Experience), es el puente definitivo entre la abstracción de las bases de datos y la codificación final en React.

El objetivo principal de esta semana fue utilizar herramientas de diseño vectorial avanzado (Figma) para crear "Wireframes" (esquemas de baja fidelidad) y, posteriormente, "Mockups" de alta fidelidad. Estos diseños no solo sirven para que la artista valide la estética de su futura plataforma, sino que actúan como un mapa de ruta estricto (Handoff) para la fase de programación, determinando qué componentes de React deberán crearse y cómo interactuarán entre sí.

- Metodología de Diseño: Enfoque "Mobile-First" y "Atomic Design":

Dada la naturaleza del tráfico web en el sector del arte contemporáneo (donde más del 70% de las visitas provienen de redirecciones desde redes sociales como Instagram o TikTok), se adoptó una filosofía de diseño "Mobile-First" (Móvil Primero). En lugar de diseñar para pantallas grandes de escritorio y luego "encoger" el contenido (Degradación Grácil), se diseñó primero la experiencia para la pantalla más restrictiva (Smartphone). Una vez resuelta la ergonomía en móvil, el diseño se expandió progresivamente (Mejora Progresiva) para adaptarse a Tablets y monitores de formato Ultra-Wide.

Para estructurar los elementos visuales, se aplicó la metodología "Atomic Design" (Diseño Atómico) postulada por Brad Frost. En lugar de diseñar páginas completas de golpe, se diseñaron primero los "Átomos" (botones

de compra, etiquetas de precio, inputs de formulario). Estos átomos se combinaron para formar "Moléculas" (una tarjeta de producto que contiene la imagen de la obra, el título y el botón). A su vez, estas moléculas crearon "Organismos" (la cuadrícula completa de la galería), garantizando una consistencia visual absoluta y preparando el terreno para la creación de componentes reutilizables en React.

- Arquitectura de la Interfaz Pública (Frontoffice Anónimo):

La interfaz destinada al público general se diseñó buscando minimizar la carga cognitiva del usuario, cediendo todo el protagonismo a la obra plástica. Se definieron las siguientes vistas clave:

1. Landing Page (Página de Inicio): Se diseñó un "Hero Section" inmersivo que ocupa el 100% del alto de la pantalla (100vh), mostrando una fotografía de alta resolución de la obra más representativa de la artista, acompañada de un texto declarativo (Statement) muy breve y un botón de llamada a la acción (Call to Action) primario: "Explorar Colección".

2. Galería de Obras (Catálogo): Para la disposición de los cuadros, se descartó la clásica cuadrícula rígida (Grid tradicional) en favor de un diseño tipo "Masonry" (Mampostería asimétrica). La justificación de esta decisión UI es puramente artística: las obras de arte tienen diferentes relaciones de aspecto (verticales, apaisadas, cuadradas). Forzar su visualización dentro de cuadrados perfectos obligaría a recortar las imágenes (Crop), destruyendo la composición original concebida por la artista. El diseño Masonry permite

encajar las imágenes manteniendo sus proporciones originales, emulando la sensación orgánica de una pared de exposición real.

3. Vista de Detalle de la Obra (Product Page): Al pulsar sobre un cuadro, el usuario accede a su ficha técnica. Se implementó un diseño de doble columna en escritorio (y apilado en móvil). A la izquierda, un visor interactivo (Lightbox) que permite hacer un zoom profundo para examinar la textura del lienzo. A la derecha, un panel "Sticky" (pegajoso) que acompaña al usuario al hacer scroll, mostrando el precio, las dimensiones, si la obra está enmarcada y el botón negro y contundente de "Añadir al Carrito" o "Comprar Ahora", aplicando la Ley de Fitts para asegurar que el área táctil del botón sea lo suficientemente grande e invitadora.

4. Flujo de Pago (Guest Checkout): La pantalla de pagos se diseñó bajo el principio de "Aislamiento". Se eliminaron intencionadamente las distracciones, ocultando el menú de navegación principal y el pie de página (Footer). El usuario solo ve el resumen de su compra, los campos para introducir su dirección de envío internacional y el formulario encriptado de Stripe para su tarjeta. Este patrón de diseño UX reduce drásticamente la tasa de carritos abandonados.

- Arquitectura de la Interfaz Privada (Frontoffice VIP y Comunidad):

Para justificar el esfuerzo que supone registrarse, la interfaz privada debía transmitir un salto cualitativo hacia una experiencia "Premium".

1. Muros de Autenticación (Modals): Los formularios de inicio de sesión y registro se diseñaron como ventanas modales minimalistas, exigiendo únicamente nombre, correo electrónico y contraseña. Se incluyó retroalimentación visual inmediata (Micro-interacciones) para la validación de contraseñas seguras.

2. El "Dashboard" del Coleccionista: Una vez logueado, el usuario es recibido en un panel de control limpio. Aquí se diseñó un "Feed" de noticias similar a una red social de alto standing. Las obras que se encuentran en estado de "Preventa VIP" se distinguen visualmente de las públicas mediante etiquetas sutiles (Badges) doradas o grises que indican "Early Access" (Acceso Anticipado), reforzando el sesgo psicológico de la escasez y la exclusividad.

- Arquitectura de la Interfaz de Administración (Backoffice de la Artista):

La herramienta de trabajo interna de la artista debía ser funcional, indestructible y altamente eficiente, alejada de la poética visual de la parte pública.

1. Gestor de Inventario: Se diseñaron tablas de datos densas que muestran el ID de la obra, miniatura, precio, stock y estado. Se incluyeron campos de búsqueda y filtros rápidos.

2. Formularios de Carga (Drag & Drop): Para subir nuevas obras, se prototipó una zona de "Arrastrar y Soltar", donde la artista puede arrastrar múltiples fotografías desde el escritorio de su ordenador. Se añadieron barras de progreso simuladas para indicar al usuario que las imágenes se están optimizando y subiendo al "Storage" (Almacenamiento) en la nube.

3. Editor de Noticias (Rich Text Editor): Se diseñó un espacio de escritura similar a plataformas como Medium, donde la creadora puede redactar artículos, insertar vídeos de su estudio y marcar mediante un simple interruptor (Toggle UI) si dicha publicación será visible para todos o solo para la comunidad de usuarios registrados.

- Transiciones y Estados de Carga (Loading States):

Un aspecto vital de la Experiencia de Usuario (UX) documentado en esta fase fue el diseño de los tiempos de espera. Al ser una aplicación construida en React JS que debe ir al servidor (Supabase) a buscar la información, existen milisegundos de latencia. En lugar de diseñar los clásicos y aburridos círculos de carga (Spinners), se diseñaron "Skeleton Screens" (Pantallas Esqueleto): bloques grises con una animación de pulso que imitan la forma que tendrán las obras y los textos antes de que la información real termine de descargarse, dando al usuario la sensación psicológica de que la página carga muchísimo más rápido.

17-octubre-2025 / Desarrollo de la Estructura de la Base de Datos

- Contexto, Importancia y Objetivos Fundamentales:

Una vez definida la capa de presentación (Frontend) y validados los prototipos visuales de alta fidelidad, el proyecto se adentró en su fase más crítica a nivel de ingeniería de software: el diseño y estructuración de la capa de persistencia de datos (Backend y Base de Datos). Una interfaz gráfica en React JS, por muy optimizada que esté, no es más que un cascarón vacío e inerte si no está conectada a un cerebro central que almacene, procese y sirva la información de manera segura y eficiente.

El objetivo principal de esta etapa fue definir el paradigma de almacenamiento y establecer los cimientos de la arquitectura de la información. Al tratarse de una plataforma de comercio electrónico de alto valor (High-Ticket E-commerce) orientada a la venta de obras de arte únicas, el margen de error es absolutamente nulo. El sistema debía ser capaz de registrar transacciones económicas, gestionar historiales de clientes, controlar un inventario estricto de unidades físicas (donde el stock siempre es 1 para una obra original) y almacenar los artículos de la zona comunitaria VIP, todo ello garantizando la máxima integridad referencial.

- El Debate Arquitectónico: Paradigma Relacional (SQL) vs. No Relacional (NoSQL):

La primera gran decisión técnica consistió en evaluar qué motor de base de datos era el idóneo para la naturaleza de este modelo de negocio. Se realizó una auditoría comparativa entre bases de datos orientadas a documentos (NoSQL, como MongoDB o Firebase Firestore) y bases de datos estructuradas en tablas (SQL, como MySQL o PostgreSQL).

1. Descarte del Paradigma NoSQL (Firebase / MongoDB):

Aunque las bases de datos NoSQL ofrecen una flexibilidad extrema y una curva de aprendizaje muy rápida para desarrolladores Frontend, se determinó que su naturaleza no estructurada y desnormalizada representaba un riesgo crítico para una pasarela de pagos. En el comercio electrónico, los datos están intrínsecamente conectados: un "Usuario" realiza un "Pedido", y ese pedido contiene una "Obra", la cual a su vez pertenece a una "Colección". Si se utilizara NoSQL, replicar estos datos en diferentes colecciones de documentos podría llevar a anomalías de actualización. Por ejemplo, si la artista decide modificar el precio o el título de un cuadro, habría que buscar y actualizar ese dato en múltiples documentos dispersos, corriendo el riesgo de generar inconsistencias graves frente a una auditoría contable.

2. Elección del Paradigma Relacional y PostgreSQL:

La decisión unánime recayó sobre el modelo relacional (SQL), seleccionando concretamente PostgreSQL. Este motor open-source está considerado como la base de datos relacional más avanzada del mundo. Su rigidez estructural es, paradójicamente, su mayor virtud para este proyecto. Obliga a definir esquemas de datos estrictos (Data Schemas) y tipos de datos fuertemente tipados (VARCHAR, BIGINT, UUID, BOOLEAN).

Más importante aún, PostgreSQL garantiza el cumplimiento estricto de las propiedades transaccionales ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). La Atomicidad asegura que una transacción

comercial es indivisible: si un cliente paga un cuadro, la base de datos registra el pago y descuenta la obra del inventario de forma simultánea. Si el servidor se cae en el milisegundo intermedio entre ambas acciones, la operación entera se revierte (Rollback), impidiendo que se cobre a un cliente sin asignarle su obra de arte. Esta robustez es innegociable al integrar la API financiera de Stripe.

- Adopción del Ecosistema Supabase (Backend-as-a-Service):

Levantar y mantener un clúster de servidores PostgreSQL tradicionales requeriría la configuración de máquinas virtuales, balanceadores de carga, cortafuegos y tareas continuas de mantenimiento DevOps (copias de seguridad, actualizaciones de parches de seguridad). Para evitar desviar recursos técnicos del objetivo principal (vender arte), se optó por una arquitectura "Serverless" mediante la adopción de Supabase.

Supabase actúa como una capa de abstracción sobre PostgreSQL. Proporciona toda la potencia bruta de una base de datos relacional, pero gestionada íntegramente en la nube. Además, auto-genera instantáneamente una API RESTful (PostgREST) a partir de las tablas creadas, permitiendo que la aplicación Frontend en React pueda realizar consultas directas (Consultas de selección, inserción o borrado) de manera segura sin necesidad de programar y mantener un servidor intermedio en Node.js o Java (Spring Boot), reduciendo drásticamente el "Time-to-Market" (tiempo de lanzamiento) de la galería.

- Desafíos de Arquitectura y Soluciones Implementadas:

Durante el modelado conceptual, surgieron varios retos logísticos y de rendimiento que debieron ser resueltos desde el diseño de la estructura de datos:

1. Gestión de Archivos Multimedia Pesados (Fotografías en Alta Resolución):

Desafío: Almacenar imágenes en formato binario (BLOB - Binary Large Object) directamente dentro de las celdas de una base de datos relacional es una práctica obsoleta que engorda exponencialmente el tamaño de la base de datos, colapsando su rendimiento en las búsquedas.

Solución de Diseño: Se determinó el uso de "Buckets" de almacenamiento de objetos (Supabase Storage, compatible con la tecnología S3 de Amazon). Las fotografías físicas de los lienzos se subirán a estos Buckets, optimizadas y procesadas por la CDN. La base de datos PostgreSQL únicamente almacenará una cadena de texto (VARCHAR) que contendrá la URL pública de dicha imagen. Esto mantiene la base de datos extremadamente ágil y ligera.

2. Prevención de Colisiones y Condiciones de Carrera (Race Conditions):

Desafío: Tratándose de cuadros originales de disponibilidad única (Stock = 1), existe el riesgo estadístico de que dos coleccionistas en distintas partes del mundo intenten comprar y pagar la misma obra exactamente en el mismo segundo.

Solución de Diseño: Se estructuró un sistema de control de concurrencia optimista a nivel de base de datos. Se definió un campo de estado booleano o enumerado (ej. 'disponible', 'reservado', 'vendido'). En el momento en que un usuario inicia el proceso de Checkout en Stripe, la base de datos marca temporalmente el registro como 'reservado' mediante un bloqueo de fila (Row-Level Lock) durante 15 minutos. Si el pago no se completa en esa ventana de tiempo, un cronjob (tarea programada) o función asíncrona libera la obra devolviéndola al estado 'disponible'.

3. Criptografía y Ofuscación de Identificadores:

Desafío: Si se utilizan identificadores numéricos incrementales (ID 1, ID 2, ID 3) para registrar a los usuarios o a los pedidos, un atacante o la competencia podría deducir fácilmente cuántas ventas está teniendo la artista simplemente creando una cuenta y viendo que su número de pedido es el 150.

Solución de Diseño: Se descartó el uso de variables "SERIAL" o "BIGINT" auto-incrementales para las claves primarias (Primary Keys) de las tablas más sensibles. En su lugar, se estandarizó el uso de UUIDs v4 (Identificadores Únicos Universales). De esta forma, el número de pedido de un cliente será una cadena alfanumérica criptográficamente segura e inescrutable (ej. 550e8400-e29b-41d4-a716-446655440000), garantizando la máxima privacidad y confidencialidad de las métricas de negocio.

24-octubre-2025 / Definición de Modelo Relacional de la Base de Datos

- Contexto y Objetivos del Esquema Físico:

Tras haber tomado la decisión arquitectónica de utilizar PostgreSQL como motor relacional (gestionado a través de Supabase), la quinta semana del proyecto se dedicó íntegramente a la creación del esquema final de la base de datos (Esquema físico) e implementación de la normalización matemática de los datos.

El objetivo principal de esta fase de ingeniería de datos fue traducir las entidades conceptuales descubiertas durante la toma de requisitos (Usuario, Obra, Pedido, Noticia) en tablas físicas reales. Este proceso no consiste simplemente en crear listados de datos, sino en establecer las reglas de negocio directamente en el núcleo del motor de la base de datos mediante restricciones (Constraints), claves primarias (Primary Keys) y claves foráneas (Foreign Keys). De este modo, se garantiza de forma absoluta la integridad referencial del sistema, impidiendo, por ejemplo, que se pueda registrar el pago de un cuadro que ya no existe en el inventario o que un pedido quede huérfano si el usuario decide eliminar su cuenta.

- Proceso de Normalización (Hasta la Tercera Forma Normal - 3FN):

Para estructurar la información de manera óptima, se procedió a modelar las entidades aplicando hasta la Tercera Forma Normal (3FN). Este estándar de diseño de bases de datos relacionales es de obligado cumplimiento para evitar la redundancia de datos (duplicidad innecesaria que consume almacenamiento) y prevenir anomalías graves de inserción, actualización o borrado.

Aplicar la 3FN significó asegurar que cada columna de una tabla dependiera única y exclusivamente de la clave primaria de esa misma tabla. Por ejemplo, en lugar de guardar la dirección de envío del usuario repetida miles de veces en cada fila de sus pedidos, la dirección se guarda o bien en el perfil del usuario, o bien vinculada de forma atómica a un pedido concreto, separando los conceptos transaccionales de los conceptos de identidad.

- Diccionario de Datos y Definición Estructural de las Tablas:

A continuación, se detalla el esquema físico implementado, desglosando las tablas maestras y transaccionales que componen el ecosistema de la Galería de Arte:

1. Tabla Maestra de Autenticación y Perfiles (`usuarios`):

Esta tabla actúa como puente entre el sistema criptográfico de autenticación de Supabase (Auth) y los datos públicos o de envío del usuario.

- id (UUID, Clave Primaria): Identificador único universal. Se vincula de forma transparente con el 'auth.uid()' del sistema para garantizar el aislamiento de los datos.

- email (VARCHAR, Unique): Correo electrónico del coleccionista, fundamental para el inicio de sesión y la recepción de recibos de Stripe.

- nombre_completo (VARCHAR): Nombre para la personalización de la experiencia en la zona VIP y la facturación.

- rol_acceso (VARCHAR): Define el nivel de privilegios. Puede ser 'ADMIN' (para la artista) o 'VIP_USER' (para el coleccionista registrado).

- fecha_registro (TIMESTAMP): Marca de tiempo automática para analizar métricas de crecimiento de la comunidad.

2. Tabla Maestra de Agrupación (`colecciones`):

Tabla diseñada para categorizar las obras. Si la artista lanza una nueva serie temática (ej. "Retratos de Primavera 2026"), se crea una única fila aquí.

- id (UUID, Clave Primaria): Identificador de la colección.

- titulo_coleccion (VARCHAR): Nombre de la serie artística.

- descripcion_conceptual (TEXT): Texto curatorial que explica la inspiración detrás de la colección.

- fecha_lanzamiento (TIMESTAMP): Permite programar lanzamientos. Las obras vinculadas a una colección con fecha futura no se mostrarán en el Frontend público, facilitando la mecánica de "Preventa VIP".

3. Tabla Central de Inventario (`obras`):

El núcleo del catálogo de la plataforma. Contiene todos los metadatos necesarios para la exhibición y venta del arte.

- id (UUID, Clave Primaria): Identificador único de la pieza física.

- coleccion_id (UUID, Clave Foránea): Relaciona la obra con su colección contenedora. Se configuró con una regla `ON DELETE SET NULL` para que,

si se borra una colección, las obras no desaparezcan, sino que pasen a estar "Sin Categorizar".

- titulo_obra (VARCHAR): Nombre de la creación.

- técnica_y_soporte (VARCHAR): Ej. "Óleo sobre lienzo", "Acrílico sobre tabla".

- dimensiones_cm (VARCHAR): Datos espaciales de la pieza física (ej. "100x120").

- precio_eur (DECIMAL / NUMERIC): Valor económico de la obra. Se implementó una restricción de comprobación en la base de datos (restricción CHECK) para garantizar que el precio nunca pueda ser negativo o cero por un error de tecleo de la artista en el panel de control (precio >= 0).

- url_imagen_principal (VARCHAR): Enlace al "Bucket" de almacenamiento donde reside la fotografía en alta resolución.

- estado_disponibilidad (VARCHAR): Campo crítico de control de stock. Acepta únicamente tres valores: 'DISPONIBLE', 'RESERVADO' (durante el proceso de checkout en Stripe) o 'VENDIDO'.

4. Tabla de Contenido Comunitario (`noticias`):

Estructura encargada de soportar el blog privado y la comunicación con la audiencia.

- id (UUID, Clave Primaria).

- titulo_post (VARCHAR).

- contenido_html (TEXT): Almacena el cuerpo de la noticia enriquecido (negritas, enlaces, cursivas) generado desde el editor del Backoffice.

- es_restringida (BOOLEAN): Interruptor binario de seguridad. Si es `TRUE` (verdadero), el motor RLS (Row Level Security) de PostgreSQL interceptará cualquier petición de un usuario no logueado y le denegará la lectura, bloqueando el acceso al contenido exclusivo.

- autor_id (UUID, Clave Foránea): Vinculado a la tabla `usuarios` (solo a aquellos con rol 'ADMIN').

5. Tablas Transaccionales (`pedidos` y `lineas_pedido`):

Para mantener la integridad financiera, se separó la cabecera del pedido del detalle del mismo (arquitectura Cabecera-Detalle), preparando el sistema para el futuro por si la artista decide vender impresiones múltiples o merchandising en el mismo carrito.

Cabecera (`pedidos`):

- id (UUID, Clave Primaria): Identificador de la factura.

- usuario_id (UUID, Clave Foránea, Nullable): Apunta al comprador. Se permite que sea nulo (NULL) para soportar a los clientes que realizan un "Guest Checkout" (compra sin registro).

- email_contacto_invitado (VARCHAR): Para enviar la factura a quienes no crearon cuenta.

- stripe_session_id (VARCHAR, Unique): ID de trazabilidad devuelto por los servidores de Stripe.

- total_pagado (DECIMAL).

- estado_transaccion (VARCHAR): 'PENDIENTE', 'PAGADO', 'ENVIADO', 'CANCELADO'.

Detalle (`lineas_pedido`):

- id (UUID, Clave Primaria).

- pedido_id (UUID, Clave Foránea): Vinculado con regla `ON DELETE CASCADE`. Si por motivos legales se elimina un pedido (borrado de datos LOPD), se eliminan automáticamente sus líneas asociadas.

- obra_id (UUID, Clave Foránea): Apunta al cuadro comprado. Se estableció una restricción `RESTRICT`. Es decir, la base de datos prohíbe terminantemente a la artista borrar una obra de arte si esta ya ha sido vendida y figura en una línea de pedido histórico, garantizando así la inmutabilidad de los registros contables.

- precio_historico_congelado (DECIMAL): Vital a nivel contable. Guarda el precio exacto al que se vendió la obra en ese milisegundo concreto, ignorando si la artista decide cambiar el precio del cuadro en el catálogo principal semanas después de la venta.

- Conclusiones del Modelado:

El esquema relacional resultante demostró ser extremadamente sólido. Al delegar la lógica de negocio (como las restricciones CHECK y los bloqueos de estado) directamente al motor PostgreSQL, se liberó al código Frontend en React de tener que realizar validaciones repetitivas y redundantes. La

base de datos se erige como la fuente de verdad absoluta e inquebrantable de la Galería de Arte, preparada para procesar transacciones sin miedo a inconsistencias estructurales.

31-octubre-2025 / Presentación a la Empresa de Interfaces y Base de Datos

- Contexto y Objetivos de la Reunión de Validación:

Al finalizar el mes de octubre, con el prototipado visual completado y la arquitectura de datos modelada, se alcanzó el primer gran hito de control del proyecto (Milestone). El objetivo primordial de esta jornada fue realizar una validación cruzada con el cliente final (en este caso, la artista creadora) y la tutoría académica. Era fundamental asegurar que toda la planificación técnica, los diagramas de software y las decisiones de diseño UI/UX cubrían de manera exacta las expectativas de negocio y las necesidades operativas de la galería online.

Presentar una arquitectura de software compleja a un perfil puramente creativo y no técnico requiere un esfuerzo considerable de traducción y abstracción. El reto no era explicar cómo funciona una clave foránea en PostgreSQL, sino demostrar cómo esa arquitectura protegería sus ventas, su inventario y la exclusividad de su obra.

- Actividades Realizadas: Defensa de la Capa de Presentación (UI/UX):

La primera mitad de la sesión consistió en una demostración interactiva. Se organizó una defensa técnica de los prototipos navegables construidos previamente en Figma.

1. Simulación del "Customer Journey": Se guio a la artista a través del flujo exacto que experimentaría un coleccionista. Se navegó desde la "Landing Page" pública, demostrando el comportamiento fluido de la cuadrícula asimétrica (Masonry Grid), hasta llegar a la pasarela de pago (Guest Checkout).

2. Justificación Estética: Se defendió la elección del "Cubo Blanco" (fondos alabastro y tipografía Serif/Sans-Serif) argumentando que esta neutralidad visual eleva el valor percibido de sus obras, emulando la experiencia de una galería de Bellas Artes tradicional. La artista validó positivamente el alto contraste y el uso agresivo del espacio negativo (White Space).

3. Demostración de la Zona VIP: Se mostró el flujo de inicio de sesión y la interfaz del "Dashboard" privado, donde los coleccionistas registrados consumirán las noticias exclusivas. Se evidenció cómo esta segmentación visual genera un fuerte incentivo para que los usuarios anónimos dejen su correo electrónico, construyendo así la base de datos de marketing de la artista.

- Actividades Realizadas: Defensa de la Capa de Persistencia y Lógica de Negocio:

La segunda mitad de la reunión tuvo un corte estrictamente técnico y logístico. Se organizó una defensa técnica de los diagramas Entidad-Relación (Modelo Relacional de Base de Datos).

1. Escalabilidad del Catálogo: Se explicó cómo la tabla maestra de 'colecciones' vinculada a las 'obras' permite una escalabilidad absoluta. La artista comprendió que podrá subir cientos de cuadros en los próximos años sin que la plataforma sufra degradación de rendimiento.

2. Resolución de Condiciones de Carrera (Concurrencia): Se abordó la mayor preocupación logística de la creadora: "¿Qué pasa si dos personas intentan comprar mi último cuadro original a la vez?". Se le explicó el mecanismo de control de concurrencia optimista diseñado en PostgreSQL, donde el sistema bloquea temporalmente el estado de la obra (pasando a 'reservado') en el momento en que el primer cliente abre la pasarela de Stripe, impidiendo pagos duplicados y devoluciones incómodas.

3. Demostración del Backoffice (CMS): Se le presentaron los "Wireframes" (esquemas estructurales) de su futuro panel de administración. Se validó que el formulario para subir obras, marcar precios y redactar noticias fuera lo suficientemente intuitivo como para que ella pudiera autogestionar su negocio digital sin requerir soporte técnico constante.

- Resultados, Feedback y Aprobación Final (Sign-off):

Tras la extensa revisión de los flujos de interacción y las reglas de negocio, los resultados de la reunión fueron sumamente positivos. Se obtuvo luz verde (aprobación formal o "Sign-off") por parte de la creadora tras

demostrar, de forma empírica y visual, que la base de datos era completamente escalable y segura.

Asimismo, se ratificó que la interfaz propuesta cumplía holgadamente con los principios de elegancia requeridos por el sector del arte, al mismo tiempo que garantizaba la inclusión digital mediante una navegación sin barreras cognitivas.

* Ajustes Menores (Feedback): Como único punto de mejora solicitado por la artista durante la presentación, se acordó añadir un campo opcional de "Certificado de Autenticidad" en la ficha de las obras de mayor valor. Este campo requerirá una ligera modificación en la tabla 'obras' de la base de datos (añadiendo una columna booleana 'incluye_certificado_fisico') para informar al comprador de que recibirá un documento firmado a mano junto con el lienzo.

* Cierre de la Fase de Diseño: Con esta aprobación unánime, se dio por clausurada la fase conceptual y de arquitectura base del proyecto. El diseño quedó "congelado" (Design Freeze), estableciendo una base inamovible y segura para iniciar la selección de tecnologías definitivas y la inminente escritura de código fuente en el mes de noviembre.

7-noviembre-2025 / Elección de Tecnologías Para Utilizar

- Contexto y Objetivos de la Arquitectura de Software:

Una vez validados los requisitos funcionales, el diseño visual de la interfaz y el modelo relacional de la base de datos, el proyecto alcanzó su punto de inflexión técnico: la definición del "Stack" Tecnológico (conjunto de herramientas, librerías y lenguajes de programación).

El objetivo fundamental de esta fase fue seleccionar un ecosistema de tecnologías que garantizara escalabilidad, seguridad transaccional, tiempos de respuesta óptimos para el tratamiento de imágenes pesadas y, sobre todo, un mantenimiento a largo plazo viable y económico para la artista. Al tratarse de un "E-commerce" que procesará cobros reales por obras de alto valor, las decisiones tomadas en esta semana establecen los cimientos inamovibles sobre los que se codificará todo el aplicativo.

- Justificación del Frontend y la Capa de Presentación (React JS):

Para la interfaz con la que interactuará el usuario final, se determinó desde el inicio que la aplicación web debía comportarse como una Single Page Application (SPA). Al servir un único archivo HTML inicial y delegar el enrutamiento y la actualización del contenido a JavaScript, se elimina por completo el parpadeo blanco de recarga entre páginas. Esta "cero fricción" es vital para mantener al comprador inmerso en la galería.

Se evaluaron los tres grandes "frameworks" del mercado: Angular, Vue.js y React.

1. Descarte de Angular: A pesar de ser un framework empresarial robusto respaldado por Google, su curva de aprendizaje, su arquitectura fuertemente acoplada a TypeScript estricto y su patrón MVC tradicional resultaban ser un "overkill" (exceso de ingeniería) para las necesidades ágiles de esta galería. El peso del "bundle" (paquete final de código) penalizaría la carga en dispositivos móviles.

2. Descarte de Vue.js: Aunque es una opción progresiva y extremadamente ligera, se descartó a favor de React debido a la madurez de su ecosistema de terceros. La integración de pasarelas de pago complejas exige librerías oficiales muy testeadas (como Stripe.js y Stripe Elements), las cuales tienen un soporte nativo y de primer nivel en la comunidad de React.

3. Elección de React (v18): La decisión recayó sobre la librería de Meta (Facebook) por tres pilares técnicos irrefutables:

- El "Virtual DOM": A diferencia de la manipulación tradicional del DOM, React crea una copia virtual en memoria. Cuando un usuario interactúa (por ejemplo, añadiendo una obra al carrito), React compara el estado anterior con el nuevo mediante su algoritmo de "Reconciliation" y actualiza únicamente el nodo exacto que ha cambiado (el icono del carrito). Esto garantiza un rendimiento gráfico extremo.

- Componentes Funcionales: Permite encapsular la interfaz en piezas de lego reutilizables (ej. <ArtworkCard />, <CheckoutButton />, <VIPNewsFeed />), facilitando enormemente la mantenibilidad del código.

- React Hooks: El uso de Hooks (como 'useState' para gestionar si un modal está abierto o cerrado, 'useEffect' para hacer llamadas asíncronas a la base de datos, y 'useContext' para mantener el estado global del carrito

de compras en toda la web) permite escribir un código declarativo, limpio y predecible.

- Justificación del Sistema de Estilos y Maquetación (Tailwind CSS):

Tradicionalmente, el diseño web implicaba separar la estructura (HTML) de la presentación (archivos CSS monolíticos), utilizando metodologías de nomenclatura complejas como BEM (Block Element Modifier). Este enfoque genera hojas de estilo kilométricas y difíciles de mantener, donde borrar una clase puede romper el diseño de otra página sin previo aviso.

- Enfoque "Utility-First": Se adoptó Tailwind CSS de forma unánime. En lugar de inventar nombres semánticos para cada contenedor de un cuadro, se utilizan clases de utilidad atómicas directamente en el código de React (ej. 'flex', 'grid-cols-3', 'bg-neutral-900', 'hover:scale-105').

- Ahorro y Rendimiento: Esta decisión redujo el tiempo de maquetación en más de un 50%. Además, Tailwind incluye un motor JIT (Just-In-Time) que purga agresivamente todo el código CSS no utilizado durante el proceso de compilación (Build). El resultado es un archivo de estilos minificado de apenas unos pocos kilobytes, acelerando drásticamente el tiempo de carga (LCP) de la galería.

- Diseño "Mobile-First" Nativo: Las variantes responsivas de Tailwind (ej. 'md:flex', 'lg:grid') permitieron programar la adaptabilidad para móviles, tablets y monitores gigantes sin escribir ni una sola "Media Query" manual.

- Justificación del Backend y Base de Datos (Supabase vs Firebase):

El segundo gran debate arquitectónico fue la capa de persistencia y la lógica del servidor. Desarrollar un Backend monolítico propio (usando Node.js/Express, PHP o Java Spring Boot) implicaba configurar servidores, programar docenas de "Endpoints" REST desde cero, gestionar la encriptación de contraseñas y asumir costes de infraestructura mensual (DevOps) que penalizarían la viabilidad del proyecto para la artista.

- Modelo BaaS (Backend-as-a-Service): Se decidió delegar la infraestructura a un servicio gestionado en la nube.

- Descarte de Firebase: Aunque es el líder del mercado, su base de datos (Firestore) es NoSQL (orientada a documentos). Como se justificó en fechas anteriores, gestionar carritos de compra, control de stock exacto y facturas cruzadas en NoSQL es ineficiente y muy propenso a errores de inconsistencia de datos.

- Elección de Supabase: Se erigió como la solución perfecta. Ofrece la misma velocidad de desarrollo que Firebase, pero montado sobre una base de datos puramente relacional (PostgreSQL). Supabase proporciona tres microservicios vitales en un solo paquete:

1. Base de Datos Relacional con API autogenerada (PostgREST) para consultar las obras desde React.

2. Sistema de Autenticación (Supabase Auth) para gestionar el registro, login y tokens JWT de la comunidad VIP de coleccionistas.

3. Almacenamiento (Supabase Storage) para alojar los archivos pesados de las fotografías en alta resolución, librando a la base de datos de esa carga binaria.

- Justificación de la Pasarela de Pagos (Stripe):

Para el procesamiento del dinero, se descartaron alternativas tradicionales como Redsys (por su pésima documentación técnica y nula experiencia de usuario) o PayPal estándar (que saca al usuario de la web rompiendo la inmersión).

- Elección de Stripe: Es el estándar de oro en la industria del comercio electrónico moderno. Se eligió por su impecable API, su entorno de pruebas (Test Mode) para simular transacciones durante el desarrollo, y su sistema de "Webhooks". Cuando un coleccionista paga, Stripe envía un evento asíncrono (Webhook) de forma segura al servidor (Supabase Edge Functions) confirmando que el dinero ha llegado al banco, momento en el cual el sistema marca la obra de arte automáticamente como 'Vendida', cerrando el ciclo logístico sin intervención humana. Además, Stripe asume toda la responsabilidad legal del cumplimiento PCI-DSS para el manejo de las tarjetas.

- Justificación de la Infraestructura de Despliegue (Vercel):

Finalmente, la aplicación compilada debía alojarse en algún lugar accesible públicamente.

- Elección de la Arquitectura Jamstack y Vercel: Nuestra aplicación Frontend, al ser compilada mediante Vite, genera archivos estáticos (HTML, CSS y JS puro) que consultan datos a la API de Supabase. Desplegar esto en un servidor tradicional tipo Apache o Tomcat es ineficiente. Se seleccionó la plataforma Vercel por su integración continua (CI/CD) nativa con nuestro repositorio de GitHub.

- Red CDN Global: Vercel distribuye nuestra Galería de Arte a través de una red Edge (Content Delivery Network). Esto significa que si un coleccionista entra a la web desde Tokio, y otro desde Nueva York, la aplicación se servirá instantáneamente desde el nodo de internet geográficamente más cercano a ellos, garantizando tiempos de carga de milisegundos y una alta disponibilidad (Uptime) del 99.99%.

14-noviembre-2025 / Estructuración Inicial de Documentación y Arquitectura Base

- Contexto y Objetivos del Andamiaje Técnico (Scaffolding):

Una vez seleccionado y justificado exhaustivamente el "Stack" tecnológico (React, Tailwind, Supabase, Stripe y Vercel), la segunda semana de noviembre marcó el inicio oficial de la fase de desarrollo empírico. El objetivo fundamental de esta etapa fue traducir las decisiones teóricas en una infraestructura de proyecto real, creando el esqueleto sobre el cual se programarían todas las funcionalidades de la Galería de Arte.

Esta fase, conocida en ingeniería de software como "Scaffolding" (Andamiaje), es crítica. Una mala estructura de carpetas o una mala configuración inicial del entorno de desarrollo genera lo que se conoce como "Deuda Técnica", lo que inevitablemente ralentizará el desarrollo a medida que la aplicación crezca en complejidad al añadir las pasarelas de pago y las rutas privadas.

- Inicialización del Repositorio y Control de Versiones (Git/GitHub):

El primer paso fue inicializar el sistema de control de versiones distribuido utilizando Git. Se creó un repositorio privado en la plataforma GitHub. El uso de Git no solo es una buena práctica obligatoria en la industria para mantener un historial de cambios (Commits) y poder revertir el código en caso de fallos críticos, sino que es el requisito previo indispensable para conectar posteriormente el proyecto con el pipeline de despliegue continuo (CI/CD) de Vercel.

Se estableció el árbol de directorios raíz del proyecto, prestando especial atención a la creación de una carpeta '/docs' independiente, destinada a albergar toda la documentación técnica, diagramas UML y este mismo manual. Asimismo, se redactó un archivo 'README.md' estandarizado. Este documento vital incluye insignias dinámicas (badges) del estado del proyecto, una descripción ejecutiva de la arquitectura del e-commerce, instrucciones detalladas de instalación local (comandos de clonación) y los

scripts de ejecución, garantizando que cualquier otro desarrollador o auditor académico pueda entender y levantar el proyecto sin asistencia.

- Elección del Empaquetador y Motor de Desarrollo (Vite vs. Create React App):

Para generar el código base de la aplicación React, se descartó de forma categórica el uso de la herramienta tradicional "Create React App" (CRA) promovida históricamente por Facebook. CRA utiliza "Webpack" bajo el capó, un empaquetador basado en JavaScript que, en proyectos modernos con múltiples librerías (Tailwind, Stripe, Supabase), resulta extremadamente lento a la hora de compilar y refrescar el navegador durante el desarrollo.

En su lugar, se procedió a la inicialización del proyecto mediante la herramienta de nueva generación 'Vite'. La justificación técnica para esta elección radica en su motor de compilación subyacente: 'esbuild'. Escrito nativamente en el lenguaje de programación Go, esbuild es capaz de pre-empaquetar y compilar las dependencias entre 10 y 100 veces más rápido que Webpack. Además, Vite aprovecha los módulos ES nativos del navegador (ESM) para ofrecer un "Hot Module Replacement" (HMR) casi instantáneo. Al diseñar una galería de arte, donde el desarrollador necesita realizar ajustes de píxeles, tipografía y color visualizando el resultado en tiempo real, esta velocidad de refresco reduce drásticamente los tiempos muertos de desarrollo.

- Arquitectura de Directorios del Código Fuente (Directory Tree):

Se configuró el andamiaje interno dentro de la carpeta '/src', creando una jerarquía escalable, modular y basada en la separación de responsabilidades (Separation of Concerns):

1. '/src/components': Destinada a albergar los componentes de UI (Interfaz de Usuario) aislados, "tontos" (sin lógica de negocio compleja) y altamente reutilizables. Aquí residirán elementos como '<ArtworkCard />' (la tarjeta que muestra el cuadro), '<CheckoutButton />', y modales de inicio de sesión.

2. '/src/pages': Contiene las vistas completas de enrutamiento que componen las páginas reales de la aplicación. Se dividió lógicamente en vistas públicas (Home, PublicGallery, ArtworkDetail) y vistas privadas o protegidas (VIPDashboard, UserOrders, AdminPanel).

3. '/src/context': Carpeta dedicada a la gestión del estado global de la aplicación. Al tener un e-commerce, es inviable pasar la información de los cuadros comprados de componente en componente (anti-patrón conocido como "Prop Drilling"). En su lugar, se utilizará la 'Context API' nativa de React. Se planificó la creación de un 'AuthContext' para mantener viva la sesión del usuario (el token JWT) en toda la web, y un 'CartContext' para memorizar qué obras ha añadido el cliente a su carrito antes de proceder al pago. Se descartó el uso de librerías externas de manejo de estado como Redux por añadir una complejidad innecesaria para los requisitos actuales.

4. '/src/lib': Directorio reservado para los archivos de inicialización y configuración de clientes de servicios externos. Aquí se instanciará el cliente de conexión a la base de datos ('supabaseClient.js') y la promesa de carga de la pasarela de pagos ('stripeClient.js').

- Desafíos de Ciberseguridad Inicial y Gestión de Variables de Entorno (.env):

El mayor riesgo de seguridad en la fase de andamiaje es la fuga accidental de credenciales. El proyecto requiere conectarse a la base de datos de producción de Supabase y a la API financiera de Stripe desde el primer día. Si estas claves secretas se escriben directamente en el código fuente (Hardcoding) y se suben al repositorio público de GitHub, cualquier bot automatizado en internet podría secuestrar la base de datos o realizar cargos fraudulentos a la cuenta de Stripe de la artista.

Para mitigar este riesgo de forma absoluta, se implementó un estricto control de secretos mediante archivos de variables de entorno '.env.local'. Se definieron variables clave como 'VITE_SUPABASE_URL', 'VITE_SUPABASE_ANON_KEY', 'VITE_STRIPE_PUBLIC_KEY' y 'STRIPE_SECRET_KEY'.

Como medida de protección final, este archivo '.env.local' fue excluido inmediatamente del sistema de control de versiones añadiéndolo al archivo '.gitignore'. Esto asegura de forma taxativa que ninguna clave de API o

contraseña administrativa pueda ser comprometida o expuesta en internet, cumpliendo con los estándares básicos de SecOps (Operaciones de Seguridad).

21-noviembre-2025 / Definición de Puntos de los Manuales de Usuario y Técnico

- Contexto y Objetivos de la Documentación Final:

A medida que la fase de diseño arquitectónico y andamiaje del código llegaba a su fin, la última semana de noviembre se dedicó a la planificación estratégica de la entrega documental (Entregables). El desarrollo de software moderno no concluye cuando el código compila sin errores; requiere una documentación robusta que garantice que todo el esfuerzo de ingeniería quede debidamente registrado, empaquetado y justificado para su evaluación académica, así como para su futura transferencia tecnológica a la cliente final.

El objetivo principal de esta fase de abstracción fue establecer el esqueleto, los índices y las plantillas base de los dos documentos vitales del proyecto: el Manual Técnico (dirigido a ingenieros y evaluadores) y el Manual de Usuario (dirigido exclusivamente a la artista y administradora del sistema).

- Estructuración del Manual Técnico (Enfoque de Ingeniería y Arquitectura):

Se determinó que el Manual Técnico debía actuar como la "Caja Negra" del proyecto, desglosando y justificando de manera extensa y académica

todas las decisiones tomadas el 7 de noviembre (elección de React JS, Tailwind CSS, y las políticas de seguridad RLS de Supabase).

Para representar la lógica de negocio subyacente, se planificó la inclusión obligatoria de diagramas UML (Lenguaje Unificado de Modelado) , prestando especial atención a los flujos asíncronos complejos que involucran a terceros:

1. Diagramas de Secuencia para la Pasarela de Pagos: Se requerirá un diagrama exhaustivo que documente el flujo de "Checkout" con Stripe. Este diagrama trazará el recorrido exacto de la información: el usuario pulsa "Comprar" en el Frontend (React) -> el sistema solicita una Sesión de Pago a la API de Stripe -> el usuario introduce su tarjeta en el entorno seguro de Stripe -> Stripe procesa el cobro con el banco -> Stripe emite un evento asíncrono (Webhook) hacia el servidor de Supabase -> Supabase ejecuta una función (Edge Function) que actualiza el estado de la obra a "Vendida" y envía el recibo al cliente.

2. Diagramas de Casos de Uso y Seguridad RLS: Documentación de las barreras criptográficas. Se ilustrará cómo el motor de PostgreSQL intercepta las peticiones y evalúa el Token JWT del usuario para decidir si le muestra una "Noticia VIP" o si le deniega el acceso (Error 401 Unauthorized).

3. Diccionario de Datos Completo: Un volcado textual de todas las tablas, tipos de variables (UUID, VARCHAR, BOOLEAN) y restricciones (Constraints) implementadas en la base de datos para asegurar el inventario.

- Estructuración del Manual de Usuario (Enfoque Funcional y Capacitación):

De forma paralela y simultánea, se estructuró el Manual de Usuario bajo una premisa innegociable: debía alejarse por completo de la jerga técnica, el código y el lenguaje de ingeniería. Este documento se concibió como una guía de "Onboarding" (Aterrizaje y Capacitación) destinada exclusivamente a la artista, que será la única operadora diaria del sistema Backoffice.

Para asegurar la máxima comprensión y reducir la curva de aprendizaje de la creadora, se establecieron las siguientes directrices de redacción:

1. Formato Visual y Procedimental: El manual empleará masivamente capturas de pantalla de la interfaz final y diagramas de flujo altamente intuitivos. Se estructurará como un recetario paso a paso (ej. "Cómo subir una nueva Colección", "Cómo marcar una obra en Preventa VIP", "Cómo leer el registro de un pedido").

2. Guía de Optimización Multimedia: Se incluirá un anexo formativo vital para el rendimiento de la galería. Se le enseñará a la artista, mediante pasos sencillos, cómo debe exportar las fotografías de sus cuadros desde programas de edición (Photoshop/Lightroom), reduciendo su peso y convirtiéndolas a formatos web modernos (WebP) antes de subirlas al 'Storage' de Supabase, evitando así que ella misma sature el ancho de banda de su propia plataforma.

3. Gestión Operativa de Stripe: Se añadirá un apartado explicando cómo debe acceder al panel de control de Stripe (externo a la galería) para

transferir los fondos recaudados (Payouts) a su cuenta bancaria personal, y cómo gestionar posibles reembolsos (Refunds) en caso de que un envío internacional sufra daños y el cliente exija la devolución del importe.

- Conclusión del Primer Trimestre:

Con la definición de estos índices documentales, se dio por clausurada de forma oficial la fase de diseño y planificación (Primer Trimestre). El proyecto quedó perfectamente encapsulado, con una arquitectura tecnológica aprobada, unas interfaces validadas y una hoja de ruta documental clara, dejando el terreno completamente preparado para iniciar la fase intensiva de codificación, pruebas y despliegue a partir del mes de diciembre.

3. SEGUNDO TRIMESTRE: Desarrollo Documental, Pruebas y Despliegue

05-diciembre-2025 / Desarrollo Inicial de Manuales de Usuario y Técnico (Fase de Abstracción y Documentación)

- Contexto y Objetivos de la Fase de Documentación:

Tras consolidar la arquitectura base durante el primer trimestre y tener el núcleo del desarrollo de código avanzado (Front-end en React y Back-end en Supabase), el inicio del segundo trimestre marcó un punto de inflexión. Era absolutamente imperativo iniciar la redacción formal de los entregables y manuales del proyecto.

El objetivo principal de esta primera semana de diciembre fue realizar un ejercicio de abstracción: detener momentáneamente la escritura de código puro para documentar, estructurar y explicar la lógica que se estaba construyendo. Se procedió a dividir la información generada de forma drástica, separándola según el público objetivo final. Por un lado, se abordó el Manual Técnico, orientado estrictamente a un perfil de ingeniería de software, administradores de sistemas y tribunales de evaluación académica. Por otro lado, se comenzó a esbozar el Manual de Usuario, diseñado con un enfoque puramente funcional y operativo, destinado exclusivamente a la artista (administradora de la plataforma).

- Desarrollo Técnico: Abstracción para el Manual Técnico:

La redacción del Manual Técnico requirió una transcripción meticulosa de todas las decisiones de arquitectura de software y seguridad tomadas hasta la fecha. Se documentaron los siguientes bloques críticos:

1. Transcripción de Diagramas UML (Lenguaje Unificado de Modelado): Se utilizaron herramientas de modelado de software avanzado (como Lucidchart o Draw.io) para vectorizar los esquemas dibujados en fases previas. Se incluyeron:

- Diagramas de Casos de Uso: Representando visualmente cómo el "Actor Anónimo" interactúa con la galería pública, frente a cómo el "Actor VIP" interactúa con las rutas protegidas.

- Diagramas de Secuencia (El Flujo de Stripe): Se documentó paso a paso la compleja asincronía del pago. Se ilustró gráficamente el momento en que

el Frontend (React) envía la petición de compra, cómo Stripe genera un 'Session ID', cómo el cliente introduce la tarjeta, y cómo el servidor de Stripe devuelve un evento asíncrono (Webhook) a la base de datos para confirmar la transacción sin intervención humana.

2. Documentación del Motor RLS (Row Level Security): Se detalló exhaustivamente la configuración de seguridad perimetral implementada en PostgreSQL a través de Supabase. El manual técnico incluye ahora los scripts SQL (Data Definition Language) exactos desarrollados para proteger la información. Se documentaron políticas concretas como:

- Política de Lectura Pública (Catálogo): Instrucciones SQL que permiten a cualquier usuario no autenticado hacer un SELECT sobre la tabla 'obras' para poder ver los cuadros.

- Política de Escritura Restringida (CMS): Instrucciones SQL que vinculan el 'auth.uid()' (ID de usuario) con el rol 'ADMIN', garantizando que es criptográficamente imposible que un usuario estándar o un atacante inserte, modifique precios o borre obras del catálogo.

- Política de Aislamiento de Pedidos (Privacidad): Reglas que aseguran que un coleccionista autenticado únicamente pueda hacer SELECT sobre la tabla 'pedidos' si la columna 'usuario_id' coincide exactamente con su propio Token JWT, impidiendo que vea las compras de otros clientes.

3. Diccionario de Datos y Tipado: Se incluyó un mapeo completo de la base de datos relacional, detallando los tipos de variables nativas de PostgreSQL (UUID para identificadores únicos, TIMESTAMP WITH TIME ZONE para

fechas de compra, NUMERIC/DECIMAL para los precios en euros) y la justificación de las restricciones (Constraints) como las claves foráneas (Foreign Keys) que enlazan un pedido con una obra concreta.

- Desarrollo Técnico: Abstracción para el Manual de Usuario:

De manera simultánea, se inició la redacción del Manual de Usuario. El reto principal de este documento fue el cambio radical de registro y de lenguaje. Dado que la usuaria final (la artista plástica) carece de formación en ingeniería informática, se evitó por completo el uso de jerga técnica, acrónimos complejos o referencias a bases de datos y APIs.

1. Enfoque Visual y Ergonomía Documental: Se diseñó un manual con un enfoque altamente visual, estructurado como una guía de "Onboarding" (aterrizaje en la plataforma). El documento está plagado de capturas de pantalla reales de la interfaz del "Backoffice" (Panel de Administración) ya construida en React.

2. Guías de Flujo Operativo (Step-by-Step): Se redactaron tutoriales paso a paso utilizando listas numeradas y resaltando con flechas rojas y círculos las áreas de la pantalla donde la artista debe hacer clic. Los tutoriales principales documentados fueron:

- Gestión del Catálogo: "Cómo añadir una nueva obra pictórica", incluyendo instrucciones vitales sobre cómo optimizar el tamaño de las fotografías antes de subirlas para no ralentizar la web.

- Comunidad VIP: "Cómo redactar una noticia exclusiva", mostrando el uso del editor de texto enriquecido integrado en la plataforma.

- Logística y Pedidos: "Cómo leer una nueva venta". Se le explicó cómo el sistema le notificará visualmente de un nuevo pago exitoso, dónde encontrar la dirección postal de envío internacional introducida por el coleccionista, y cómo actualizar el estado del pedido a "Enviado" una vez haya entregado el lienzo a la empresa de mensajería (ej. DHL o FedEx), añadiendo el número de seguimiento (Tracking Number) para la tranquilidad del comprador.

12-diciembre-2025 / Análisis Arquitectónico y Opciones de Despliegue de Aplicativos

- Contexto y Objetivos del Despliegue en la Nube (Cloud Computing):

La segunda semana de diciembre marcó el inicio de la fase de operaciones e infraestructura (DevOps). El objetivo central de esta etapa fue definir de manera definitiva la infraestructura en la nube (Cloud) que alojaría la aplicación web de la Galería de Arte en su entorno de producción. El reto de diseño arquitectónico consistía en buscar el equilibrio perfecto entre el máximo rendimiento (vital para cargar un catálogo visual con imágenes de muy alta resolución sin latencia), la seguridad transaccional exigida por los pagos con tarjeta, y el coste cero de mantenimiento operativo para la creadora, permitiéndole centrarse en su arte y no en gestionar servidores.

- Desarrollo Técnico: Estudio Comparativo de Arquitecturas:

Para garantizar que la plataforma de venta pudiera soportar sin caídas posibles picos masivos de tráfico (por ejemplo, tras el lanzamiento de una colección anunciada en una campaña viral de Instagram), se realizó un estudio comparativo exhaustivo de las diferentes arquitecturas de despliegue disponibles en el mercado.

1. Descarte de Servidores Tradicionales (VPS y Hosting Compartido):

La opción de alquilar un Servidor Privado Virtual (VPS) o un alojamiento (hosting) web tradicional fue descartada de forma categórica y temprana. Desplegar una aplicación web moderna orientada a la venta transaccional en servidores clásicos (tipo Apache, Nginx o en contenedores Tomcat) requiere tareas constantes, manuales y costosas de DevOps. Este enfoque obligaría a la artista a lidiar con la instalación periódica de parches de seguridad del sistema operativo (Linux), la renovación manual de certificados criptográficos SSL, y la configuración de complejos balanceadores de carga para evitar que la web colapse si entran muchos usuarios a la vez. Esta carga técnica y económica es inviable para un modelo de negocio independiente.

2. Elección de la Arquitectura Serverless (Jamstack):

Frente al modelo tradicional monolítico, la decisión arquitectónica recayó sobre la adopción de una infraestructura "Serverless" (Sin Servidor) apoyada en la filosofía Jamstack (JavaScript, APIs y Markup). Dado que nuestra aplicación de galería es un Frontend SPA (Single Page Application) desarrollado en React y compilado eficientemente con Vite, el resultado

final del proceso de "Build" (construcción) no es un programa que deba ejecutarse en un servidor que consume recursos las 24 horas del día. En su lugar, Vite genera una carpeta con archivos puramente estáticos: HTML estructurado, CSS minificado agresivamente mediante el compilador de Tailwind, y JavaScript puro y ofuscado. Estos archivos estáticos, que pesan apenas unos pocos megabytes, son los que atacan directamente a la API RESTful autogenerada de Supabase y se comunican con los endpoints financieros de Stripe.

3. Selección de la Plataforma Vercel como Proveedor de Infraestructura:

Para dar alojamiento a estos archivos estáticos e inyectarlos en la red global de internet, se eligió la plataforma Vercel. La justificación técnica de esta decisión se cimienta en tres capacidades de grado empresarial (Enterprise-grade) que Vercel ofrece de forma accesible:

- Canal de Integración Continua (CI/CD): Se seleccionó Vercel fundamentalmente por su integración continua nativa y sin fricciones con los repositorios de código de GitHub. Esto establece un flujo de trabajo automatizado donde, cada vez que el desarrollador aprueba un cambio o una corrección de errores en la rama principal (Main branch), Vercel intercepta el evento, levanta un contenedor efímero, ejecuta las dependencias de Node.js, compila el proyecto y lo despliega en vivo en menos de un minuto. Todo esto sin tiempos de inactividad (Zero-Downtime Deployments).

- Red Edge de Entrega de Contenidos (Global CDN): A diferencia de un servidor VPS que reside físicamente en una única ciudad (ej. Madrid), Vercel distribuye de forma automática todos los archivos estáticos de la Galería de Arte a través de una inmensa red CDN (Content Delivery Network) global o "Edge Network". Esto asegura que si un coleccionista en Nueva York, otro en Tokio y otro en París acceden al sistema para intentar adquirir la misma obra de arte, todos experimentarán tiempos de carga ultrarrápidos, del orden de escasos milisegundos. Esto es posible porque la aplicación no viaja desde España hasta Japón, sino que se sirve directamente desde el nodo de internet geográficamente más cercano al cliente.

- Alta Disponibilidad y Certificados SSL Automatizados: Esta infraestructura perimetral hiperdistribuida garantiza el cumplimiento del requisito de alta disponibilidad, asegurando un 99.9% de Uptime. Además, a diferencia del alojamiento tradicional, Vercel provisiona, negocia y renueva de forma 100% automática los certificados de seguridad SSL/TLS mediante la autoridad certificadora Let's Encrypt. Esto fuerza a que todo el tráfico de la web se realice bajo el protocolo seguro HTTPS, un prerrequisito técnico y legal innegociable exigido por Stripe para poder abrir el iframe de cobro y procesar datos de tarjetas de crédito.

19-diciembre-2025 / Pruebas de Despliegue en Entorno Local (TomCat / Servidor HTTP Local)

- Contexto y Objetivos del Proceso de Compilación (Build) y Simulación:

A medida que el mes de diciembre avanzaba y el código fuente en React tomaba su forma definitiva, se hizo imperativo realizar simulacros de despliegue antes de inyectar el código en la red global. Durante la fase de desarrollo, la aplicación se ejecuta mediante un servidor de desarrollo en caliente (Hot Module Replacement) que no refleja la realidad del código final.

Antes de confiar el aplicativo al pipeline automatizado de Vercel, es una práctica obligatoria de ingeniería de software generar una compilación de producción (Production Build) en el entorno local. El objetivo principal de esta semana fue emular un servidor tradicional (simulando el comportamiento de un servidor Apache, Nginx o TomCat) dentro de la propia máquina de desarrollo para "cazar" errores de enrutamiento estático, dependencias faltantes o fallos en la minificación de los estilos de Tailwind CSS que podrían romper la interfaz de la galería y arruinar la experiencia de compra de los coleccionistas.

- Desarrollo Técnico: Compilación y Generación del Entregable (Directorio /dist):

El primer paso técnico consistió en detener el servidor de desarrollo y ejecutar el comando de compilación definitivo: 'npm run build'. Este comando invoca al empaquetador subyacente de Vite (esbuild), el cual realiza una serie de tareas críticas de optimización intensiva:

1. Tree Shaking (Agitación del Árbol): El compilador analiza todo el código JavaScript e importa solo las funciones exactas de React y de las librerías de terceros (como Stripe.js o Supabase Client) que realmente se están utilizando, eliminando todo el "código muerto" para reducir drásticamente el peso de la web.

2. Purga de CSS (JIT Compiler): El motor de Tailwind escanea todos los componentes de la interfaz y elimina cualquier clase de diseño que no se haya aplicado a las tarjetas de las obras de arte o a los menús, generando un archivo CSS minúsculo.

3. Ofuscación y Minificación (Uglify/Minify): El código JavaScript legible por humanos se comprime, eliminando espacios, comentarios y renombrando variables a letras simples (ej. de 'precioObra' a 'x'), haciendo que el código sea ilegible para atacantes (ofuscación) y extremadamente rápido de descargar para el navegador del usuario.

El resultado de este proceso es la generación automática de la carpeta '/dist' (Distribution). Esta carpeta contiene la aplicación compilada, ofuscada y lista para producción: un único archivo 'index.html' acompañado de sus correspondientes "chunks" (fragmentos) de JavaScript y CSS estáticos. Para probar esta carpeta, se levantó un servidor HTTP local estático, emulando un entorno real de internet.

- Desafíos Técnicos Identificados: El Clásico Error "404 Not Found" en SPA:

Durante la navegación de prueba sobre este servidor local estático, se detectó un fallo arquitectónico crítico inherente a todas las Single Page

Applications (SPA). Al estar utilizando la librería 'React Router DOM' para la navegación en el lado del cliente (Client-Side Routing), las transiciones entre páginas son fluidas porque JavaScript intercepta los clics y cambia la interfaz sin recargar la web.

El problema surgió al intentar simular el comportamiento de un usuario real que recarga la página. Si un coleccionista guarda en marcadores (Bookmarks) la URL de una obra específica (ej. 'localhost:3000/galeria/obra-retrato-azul') o si refresca la pestaña del navegador estando en la ruta privada de noticias VIP ('/vip/noticias'), el navegador hace una petición real al servidor HTTP pidiendo el archivo físico 'obra-retrato-azul.html' o la carpeta '/vip'. Como Vite solo ha generado un único archivo 'index.html' en la carpeta raíz, el servidor no encuentra el destino y devuelve un error fatal "404 Not Found" (Página no encontrada), bloqueando por completo al cliente.

- Resolución del Enrutamiento (Rewrite Rules) y Preparación para Vercel:

Para solucionar este fallo de routing, se tuvo que intervenir en la configuración del servidor. Se establecieron reglas de reescritura de URL (Rewrite Rules o Fallback Routing). Se configuró el servidor local estático para que interceptara cualquier petición entrante que devolviera un error 404 y, en su lugar, redirigiera silenciosamente todo el tráfico hacia el archivo raíz 'index.html'. De este modo, el servidor siempre devuelve el cascarón de la aplicación web, y es el JavaScript de React el que lee la URL del navegador y decide qué componente (qué cuadro o noticia) debe pintar en pantalla.

Esta configuración probada en local fue de vital importancia, ya que documentó la necesidad absoluta de crear un archivo de configuración nativo llamado 'vercel.json' en la raíz del proyecto. En este archivo se plasmará la regla de enrutamiento ('"rewrites": [{"source": "/(.*)", "destination": "/index.html"}]') para asegurar que, cuando la plataforma suba a la nube de Vercel en enero, la navegación de los clientes sea robusta y a prueba de recargas de página accidentales durante el proceso de compra.

- Desafíos Técnicos Identificados: Simulación de Pagos y Webhooks (Stripe CLI):

El segundo gran reto de desplegar en un entorno local es que el servidor de desarrollo no está conectado a internet público, por lo que no tiene una dirección IP accesible desde el exterior.

En un flujo de compra real, cuando un cliente paga en la pasarela, los servidores financieros de Stripe envían un aviso asíncrono (Webhook) hacia la base de datos de la Galería (Supabase Edge Functions) para confirmar que el dinero ha sido cobrado, desencadenando la actualización del stock de la obra a "Vendida". Al estar en local, Stripe no puede "encontrar" el ordenador del desarrollador para enviarle esa confirmación.

Solución: Para validar que toda la lógica contable y de base de datos funcionaba correctamente sin tener que subir el código a producción, se

instaló y configuró la herramienta oficial 'Stripe CLI' (Command Line Interface). Mediante comandos de terminal, se abrió un túnel seguro (Forwarding) que enrutaba los eventos de prueba de Stripe directamente hacia el puerto local (localhost) de la máquina de desarrollo. Esto permitió simular compras exitosas, tarjetas declinadas y fraudes con tarjetas de prueba, confirmando visualmente en la base de datos local que el estado de los pedidos y el inventario de las obras respondían de manera perfecta a los estímulos financieros antes de arriesgar el negocio en internet.

09-enero-2026 / Pruebas de Despliegue en Vercel (Producción)

- Contexto y Objetivos de la Puesta en Producción (Go-Live):

Tras haber validado exhaustivamente el comportamiento del código y la pasarela de pagos en el servidor local estático durante el mes de diciembre, la primera semana de enero de 2026 supuso el cruce del umbral definitivo: la subida oficial de la Galería de Arte a internet.

Durante esta fase no se programó nueva lógica de negocio ni se diseñaron nuevos componentes visuales de React. El esfuerzo se centró única y exclusivamente en labores de "DevOps" (Operaciones de Desarrollo) y configuración de infraestructura en la nube. El objetivo primordial fue conectar el repositorio de código alojado en GitHub con la plataforma de alojamiento Vercel, garantizando que el entorno de producción fuera un reflejo exacto y seguro del entorno de desarrollo, y estableciendo las bases para que cualquier futura actualización del catálogo por parte de la artista

se reflejara en la web sin tiempos de inactividad (Zero-Downtime Deployment).

- Desarrollo Técnico: Configuración del Pipeline CI/CD (Integración y Despliegue Continuos):

El despliegue manual de archivos mediante protocolos obsoletos como FTP (File Transfer Protocol) es una práctica desterrada en la ingeniería de software moderna por su propensión al error humano y su falta de trazabilidad. En su lugar, se implementó un flujo automatizado de Integración y Despliegue Continuos (CI/CD).

1. Autorización OAuth: Se autorizó una conexión segura (mediante el protocolo OAuth) entre el repositorio privado de GitHub y el panel de control de Vercel. Se le otorgaron permisos a Vercel para "escuchar" los eventos que ocurrieran exclusivamente en la rama principal ('main' branch) del proyecto.
2. Automatización del Flujo: Se configuró el sistema para que, a partir de este momento, cuando el desarrollador ejecute el comando 'git push' desde su terminal local hacia GitHub, Vercel intercepte este evento web y desencadene automáticamente una reacción en cadena en sus servidores sin intervención humana.

- Desarrollo Técnico: Aprovisionamiento del Motor de Base de Datos en la Nube:

De manera paralela al código Frontend, era necesario preparar el Backend. Hasta esta fecha, las pruebas se habían realizado en entornos volátiles.

1. Ejecución de Scripts DDL (Data Definition Language): Se accedió al panel de producción de Supabase y se ejecutaron los volcados de código SQL creados en octubre. Esto generó la estructura física, vacía y definitiva de las tablas 'usuarios', 'obras', 'colecciones', 'pedidos' y 'noticias'.

2. Activación del RLS en Producción: Se habilitó formalmente el motor Row Level Security (RLS) en estas tablas de producción, asegurando que desde el minuto uno la base de datos estuviera blindada contra accesos anónimos a las rutas de facturación o a las noticias VIP.

- Gestión Criptográfica de Variables de Entorno (Secrets Management):

Uno de los hitos más críticos de la ciberseguridad en el despliegue de un e-commerce es la inyección de las variables de entorno. Los archivos '.env.local' utilizados en diciembre jamás deben subirse a GitHub. Sin embargo, los servidores de Vercel necesitan conocer las claves para poder conectar React con Supabase y Stripe.

1. Inyección Segura: Se accedió a la configuración del proyecto en el panel web de Vercel (Project Settings -> Environment Variables). Aquí se introdujeron de forma encriptada las claves maestras: 'VITE_SUPABASE_URL', la clave pública 'VITE_SUPABASE_ANON_KEY', la clave pública de la pasarela 'VITE_STRIPE_PUBLIC_KEY' y, fundamentalmente, las claves secretas de webhook de Stripe.

2. Prevención de Brechas: Al inyectar las variables directamente en la infraestructura de Vercel, estas se inyectan en memoria durante el proceso de compilación, asegurando que nunca queden "hardcodeadas" (escritas a fuego) en el código fuente descargable. Si un atacante lograra inspeccionar el código compilado de la galería en su navegador, jamás encontraría la clave privada de Stripe, protegiendo a la artista de fraudes financieros o accesos no autorizados a su cuenta bancaria.

- El Proceso de Compilación en la Nube y Emisión de Certificados (SSL/TLS):

Una vez conectadas las piezas, se forzó el primer despliegue a producción. Se verificó y auditó el proceso paso a paso a través de los registros (Logs) de la terminal virtual de Vercel:

1. Aprovisionamiento: Vercel aprovisionó un contenedor Linux efímero.

2. Resolución de Dependencias: Ejecutó el comando 'npm install' para descargar las miles de dependencias subyacentes de Node.js, React, Tailwind y Stripe.

3. Compilación Vite: Ejecutó 'npm run build', empaquetando y ofuscando todo el JavaScript, y generando la carpeta estática '/dist'.

4. Redireccionamiento SPA: Vercel leyó automáticamente el archivo 'vercel.json' configurado en diciembre, aplicando las reglas de reescritura para solucionar los errores 404 en las recargas de página de nuestra Single Page Application.

5. Distribución Global (CDN): El resultado final se distribuyó a través de la red Edge de Vercel, alojando copias del código en servidores alrededor del mundo.

Finalmente, Vercel aprovisionó y emitió de manera 100% automatizada un certificado criptográfico SSL/TLS gratuito a través de la entidad Let's Encrypt. Esto levantó el protocolo HTTPS en la web, dibujando el candado de seguridad en el navegador. Este paso técnico no es opcional: el protocolo HTTPS es un requisito legal del estándar PCI-DSS; sin él, los navegadores modernos (Chrome, Safari) bloquean la ejecución del iframe de Stripe, haciendo físicamente imposible que un coleccionista introduzca los datos de su tarjeta de crédito.

- Configuración de Dominio Personalizado:

Para dotar al proyecto del nivel de exclusividad exigido por el mercado del arte contemporáneo (High-Ticket), se sustituyó el dominio genérico autogenerado por Vercel (ej. 'galeria-arte-app.vercel.app') por un dominio personalizado (ej. 'www.nombreartista.com'). Para ello, se modificaron los registros DNS (Domain Name System) tipo A y CNAME en el proveedor de dominios, apuntándolos hacia los servidores de Vercel. Tras la propagación global de las DNS, la galería quedó oficialmente inaugurada y accesible al público mundial.

16-enero-2026 / Fase de Pruebas Extensivas (QA Testing) y Corrección de Errores (Bugs)

- Contexto y Objetivos del Aseguramiento de Calidad (Quality Assurance):

Con la aplicación formalmente desplegada en los servidores de producción de Vercel y conectada a la base de datos viva de Supabase, se declaró el inicio de la fase de "Quality Assurance" (QA) o Aseguramiento de Calidad.

Lanzar un e-commerce al público general sin una batería de pruebas rigurosa es una negligencia inaceptable, especialmente en el sector del arte (High-Ticket), donde los carritos de compra manejan importes elevados. Un solo fallo en la pasarela de pagos o un error visual en el renderizado del catálogo puede destruir instantáneamente la credibilidad y la confianza de un coleccionista. Por ello, esta semana se dedicó a someter el código a un estrés sistemático, buscando deliberadamente romper la aplicación en tres frentes críticos: Interfaz y Ergonomía (UI/UX), Lógica Financiera (Stripe) y Ciberseguridad (Pen-Testing).

- Desarrollo Técnico: Pruebas de Ergonomía Visual y Rendimiento (Frontend):

Dada la altísima carga visual del proyecto, las pruebas de Frontend se centraron en la correcta visualización del material fotográfico en múltiples resoluciones (Responsive Design) y condiciones de red.

1. Auditoría de Carga (Lazy Loading y WebP): Se utilizaron las herramientas de desarrollo del navegador (Chrome DevTools) para simular una conexión 3G lenta. Se verificó empíricamente que la técnica de "Lazy Loading" (carga diferida) funcionaba: el navegador solo solicitaba al servidor las imágenes

de los cuadros que entraban en el campo visual del usuario (Viewport), ignorando los que quedaban más abajo. Esto demostró un ahorro masivo de ancho de banda.

2. Pruebas Táctiles en Dispositivos Físicos: Se navegó la galería utilizando hardware real (tablets iPad Pro y smartphones Android). Se validó que el "Lightbox" (el visor de imágenes a pantalla completa) permitiera realizar el gesto de "Pellizcar para hacer Zoom" (Pinch-to-zoom) de forma fluida, permitiendo a los coleccionistas inspeccionar el relieve y la textura del óleo sin retardos.

3. Contraste y Accesibilidad (WCAG): Se sometió la paleta de colores del "Cubo Blanco" a validadores de contraste. Se ajustaron ligeramente los grises de los precios (pasando de un text-gray-400 a un text-gray-600 en Tailwind) para garantizar que las personas con fatiga visual o en entornos de alta iluminación pudieran leer los importes sin dificultad.

- Desarrollo Técnico: Pruebas de la Lógica Financiera y Transaccional (Stripe):

El núcleo de la Galería de Arte es su capacidad de procesar cobros de forma automatizada. Se habilitó el "Modo de Prueba" (Test Mode) de Stripe para realizar simulacros de compra sin mover dinero real.

1. El "Happy Path" (Camino Feliz): Se simuló una compra perfecta utilizando la tarjeta de crédito de prueba proporcionada por Stripe. Se verificó que, tras el pago, el usuario era redirigido a la página de "Éxito",

recibía su recibo digital, y la base de datos descontaba el cuadro del inventario.

2. Pruebas de Estrés Transaccional (Unhappy Paths): La verdadera calidad del software se demuestra cuando las cosas fallan. Se introdujeron deliberadamente tarjetas de crédito marcadas por Stripe como "Fondos Insuficientes", "Tarjeta Caducada" o "Reportada por Fraude". Se comprobó que el Frontend de React capturaba la excepción financiera (Error Handling) y mostraba al coleccionista un mensaje de error elegante y tranquilizador ("Tu banco ha declinado la operación, por favor intenta con otra tarjeta"), sin que la página web colapsara o mostrara un pantallazo blanco.

3. Auditoría de Webhooks Asíncronos: Se provocó un cierre forzoso del navegador del usuario justo en el milisegundo en que pulsó "Pagar". Se confirmó que, a pesar de que el usuario cerró la web, el servidor de Stripe logró comunicarse por la puerta trasera (Webhook) con las "Edge Functions" de Supabase, registrando el pedido exitosamente. Esto certificó que la plataforma no depende de la conexión del cliente para asegurar la integridad contable.

- Desarrollo Técnico: Pruebas de Seguridad y Penetración (SecOps / Pen-Testing RLS):

Para auditar las barreras criptográficas de la base de datos (PostgreSQL Row Level Security), se simularon ataques de intrusión (Ethical Hacking).

1. Intento de Escalada de Privilegios Vertical: Se inició sesión con una cuenta de usuario VIP estándar. Desde la consola del navegador, se intentó

forzar una petición HTTP POST directa a la API de Supabase intentando cambiar el precio de una obra (ej. de 1500€ a 1€). El motor RLS de PostgreSQL bloqueó la inyección y devolvió un error de "Acceso Denegado", demostrando que solo el token asociado al rol 'ADMIN' (la artista) tiene permisos de modificación (UPDATE).

2. Intento de Escalada de Privilegios Horizontal (Brecha de Privacidad): Se intentó vulnerar el historial de pedidos. Un usuario logueado inyectó en la URL el número de pedido (UUID) perteneciente a otro cliente diferente. El sistema, al evaluar que el 'auth.uid()' del solicitante no coincidía con el 'usuario_id' del dueño de la factura, devolvió un array vacío [], garantizando el secreto fiscal y el cumplimiento absoluto del Reglamento General de Protección de Datos (RGPD).

- Análisis y Plan de Mitigación de Riesgos (Contingencias):

Durante esta fase de pruebas, se documentó un registro de riesgos y sus correspondientes estrategias de mitigación implementadas en el código:

* Riesgo 1: Pérdida de Conexión en el Checkout (Riesgo Alto).

- Descripción: El usuario pierde la red Wi-Fi o la cobertura 4G/5G justo después de introducir la tarjeta y pulsar pagar.

- Estrategia de Mitigación: Implementación de Webhooks. La facturación y el cambio de estado de la obra ("Disponible" a "Vendida") no la realiza el Frontend (React) del usuario, sino el servidor de Stripe comunicándose directamente con el Backend (Supabase). Si el usuario se desconecta, la compra se procesa igualmente y se le notifica por email.

* Riesgo 2: Exposición de Obras en Preventa (Riesgo Crítico).

- Descripción: Un usuario anónimo descubre la URL exacta de una obra oculta destinada exclusivamente a coleccionistas VIP.

- Estrategia de Mitigación: Filtros a nivel de servidor. Las rutas de React están protegidas, pero además, la consulta SQL a la tabla 'obras' filtra automáticamente cualquier registro que tenga la etiqueta 'solo_vip = true' si el usuario no aporta un Token JWT válido en la cabecera de la petición HTTP.

* Riesgo 3: Compra Simultánea del Mismo Cuadro (Condición de Carrera) (Riesgo Medio).

- Descripción: Al tener un stock de una sola unidad por cuadro original, dos coleccionistas en distintas zonas horarias introducen la tarjeta al mismo tiempo por la misma obra.

- Estrategia de Mitigación: Bloqueo optimista (Optimistic Locking) en PostgreSQL. Al generar el enlace de pago (Checkout Session) para el primer usuario, el estado de la obra pasa a "Reservado" durante un margen de caducidad de 15 minutos (TTL - Time To Live), desapareciendo temporalmente del catálogo para el resto de visitantes hasta que la sesión expire o se complete el pago.

23-enero-2026 / Revisión Final y Auditoría de Documentación

- Contexto y Objetivos de la Sincronización Documental (Single Source of Truth):

A finales de enero, tras haber superado con éxito la exigente fase de pruebas intensivas (QA Testing) y haber parcheado todos los errores menores detectados en la interfaz de React y en los webhooks de Stripe, el código fuente de la aplicación alcanzó un estado de madurez y estabilidad absolutas. Sin embargo, en la ingeniería de software moderna guiada por metodologías ágiles, es extremadamente común que el código evolucione mucho más rápido que los manuales escritos, generando un fenómeno negativo conocido como "Docs Rot" (Putrefacción de la Documentación).

El objetivo innegociable de esta semana fue detener por completo la escritura de código y dedicar todos los esfuerzos a garantizar que la documentación fuera la "Single Source of Truth" (Única Fuente de Verdad). Era vital sincronizar los esquemas, diagramas y textos redactados en los meses anteriores con la versión final y real del software (Release Candidate v1.0.0), asegurando que el tribunal evaluador y la artista tuvieran en sus manos un manual que reflejara exactamente lo que estaba subido a los servidores de Vercel.

- Auditoría Cruzada de la Arquitectura de Base de Datos y APIs:

La primera tarea de auditoría tuvo un carácter puramente técnico y estructural. Se realizó una revisión exhaustiva de la carpeta '/docs' del

repositorio, comparando los diagramas originales con el esquema físico final desplegado en Supabase.

1. Verificación del Modelo Entidad-Relación (ER): Se comprobó celosamente que cada tabla dibujada en los diagramas UML (Lenguaje Unificado de Modelado) correspondiera bit a bit con las columnas físicas de la base de datos de producción. Por ejemplo, se verificó que la tabla 'obras' incluyera el campo booleano 'incluye_certificado_fisico' que la artista solicitó durante la reunión de validación de octubre, actualizando los diagramas gráficos para que no existiera ninguna discrepancia.

2. Auditoría de los Diagramas de Flujo Financiero: Se revisaron los diagramas de secuencia que explican la integración con Stripe. Durante el desarrollo en diciembre, la lógica de los Webhooks sufrió optimizaciones para gestionar las devoluciones (Refunds) de forma más eficiente. Se procedió a redibujar el flujo lógico en el Manual Técnico para explicar con precisión meridiana cómo la plataforma de la Galería reacciona contablemente cuando la artista emite un reembolso a un coleccionista desde el panel de control de Stripe.

- Actualización de Activos Visuales en el Manual de Usuario:

El Manual de Usuario, al ser la herramienta de capacitación directa para la creadora, depende fundamentalmente de la precisión de sus capturas de pantalla. Durante la fase de pruebas de accesibilidad (WCAG) de principios de enero, la interfaz de usuario (UI) sufrió ligeras modificaciones: se aumentaron los márgenes (Paddings) de las tarjetas de los cuadros

mediante Tailwind CSS, se oscurecieron las tipografías de los precios para mejorar el contraste y se modificó el diseño de los botones transaccionales.

Para evitar cualquier confusión operativa, se desecharon todas las capturas de pantalla obsoletas tomadas en noviembre. Se procedió a realizar un nuevo reportaje fotográfico completo de la plataforma en su versión final desplegada en el dominio de producción. Se actualizaron las guías de "Paso a Paso" demostrando cómo se ve exactamente el formulario de subida de nuevas obras y cómo se previsualizan las noticias en la zona VIP de los coleccionistas.

- Preparación del Repositorio y el Archivo README.md (Handoff Técnico):

Finalmente, se auditó la puerta de entrada al proyecto para cualquier desarrollador externo o miembro del tribunal académico: el archivo README.md alojado en la raíz del repositorio de GitHub. Un proyecto excelente puede ser suspendido si el evaluador es incapaz de ejecutarlo en su máquina local debido a instrucciones deficientes.

1. Revisión de Insignias y Metadatos: Se comprobó que el README contuviera las insignias (badges) correctas indicando que el despliegue en Vercel era exitoso ('passing') y que las dependencias de React estaban actualizadas.

2. Guía de Despliegue Local (Local Environment): Se redactó un apartado hiperdetallado con las instrucciones de clonación ('git clone'). Se listaron de forma clara todas y cada una de las variables de entorno (.env) que un

evaluador necesitaría configurar para levantar el proyecto sin bloqueos. Se hizo especial hincapié en explicar cómo el tribunal debía colocar sus propias claves de prueba de Stripe ('pk_test_...') y las URL de sus propios proyectos de Supabase para poder evaluar la plataforma de e-commerce de forma totalmente funcional, independiente y segura en sus propios ordenadores.

30-enero-2026 / Preparación para la Presentación y Defensa

- Contexto y Objetivos de la Oratoria y Defensa Pública:

A medida que el mes de enero llegaba a su fin y el desarrollo técnico del aplicativo web se daba por finalizado y documentado, el enfoque del proyecto dio un giro radical de 180 grados. En el mundo de la ingeniería de software y el emprendimiento digital, un proyecto con una arquitectura excelente, un código limpio y una base de datos inquebrantable puede fracasar estrepitosamente si el equipo no sabe cómo comunicarlo.

El objetivo primordial de esta semana, por tanto, fue transformar meses de código complejo, esquemas relacionales abstractos y lógicas asíncronas de pasarelas de pago, en un discurso oral claro, persuasivo, comercializable y entendible para un tribunal evaluador que, si bien tiene un perfil técnico, necesita ser convencido de la viabilidad comercial de la plataforma "Galería de Arte Propia". Se aplicaron técnicas de oratoria, presentación de producto e ingeniería de ventas.

- Desarrollo Narrativo: Estructuración del Discurso (El Pitch Comercial):

La semana se dedicó en exclusiva a estructurar la exposición oral, diseñando un "Slide Deck" (presentación de diapositivas) limpio y minimalista, acorde a la estética del "Cubo Blanco" desarrollada para la web. El guion de la presentación se estructuró estratégicamente en cuatro actos narrativos fundamentales:

1. El Problema (Pain Point): La presentación no comenzaría hablando de React o de bases de datos, sino exponiendo la ineficiencia, el hermetismo y el abuso de las altas comisiones (frecuentemente superiores al 50%) que las galerías de arte tradicionales y los intermediarios imponen a los creadores independientes. Se enfatizó la pérdida del control de los datos de los clientes (Data Ownership) por parte del artista cuando delega sus ventas a plataformas de terceros.

2. La Solución (Propuesta de Valor): Se presentó la conceptualización de la Galería de Arte Propia como una plataforma Direct-to-Consumer (D2C). Una herramienta híbrida que no solo es un E-commerce con pagos sin fricción (Guest Checkout), sino una herramienta de construcción de comunidades (Membership) que premia la lealtad de los coleccionistas VIP.

3. La Arquitectura Técnica (Justificación): En este bloque se preparó la defensa puramente técnica. Se esquematizó de forma visual por qué se tomó la arriesgada decisión de desarrollar todo el código desde cero con React JS y Supabase, en lugar de utilizar soluciones prefabricadas (CMS) como Shopify o WordPress/WooCommerce. Se argumentó que la arquitectura Serverless proporciona a la artista un control absoluto, una personalización sin límites, tiempos de carga muy superiores y un coste de

mantenimiento fijo mensual cercano a cero, algo imposible de lograr con las suscripciones mensuales de Shopify.

4. La Demostración Comercial (El Clímax): El acto final reservado para mostrar el producto en funcionamiento, demostrando empíricamente que todos los objetivos planteados en septiembre se habían cumplido con éxito.

- Gestión de Riesgos: Plan B y Creación de Material Audiovisual (Screencast):

Uno de los mayores axiomas en la presentación de proyectos tecnológicos es que las demostraciones en vivo (Live Demos) son extremadamente propensas a fallar por factores externos que escapan al control del desarrollador (la conocida "Ley de Murphy"). Depender de la conexión Wi-Fi del centro educativo o del salón de actos para procesar un pago en tiempo real con la API de Stripe representa un "Single Point of Failure" (Punto Único de Fallo) inaceptable para una evaluación final.

Para mitigar de forma absoluta este riesgo, se planificó y ejecutó un Plan de Contingencia Audiovisual. Además de preparar el entorno real para la demostración, se invirtieron varios días en grabar y editar un vídeo en alta definición (Screencast).

Este material audiovisual, narrado y editado con cortes limpios, mostró el flujo crítico (Happy Path) de la aplicación de principio a fin, garantizando que el tribunal pudiera evaluar el funcionamiento perfecto del software bajo cualquier circunstancia:

- Se grabó la navegación fluida por la galería pública asimétrica (Masonry).

- Se grabó el proceso de inicio de sesión de un usuario VIP y el descubrimiento de una noticia exclusiva.

- Se grabó la inserción de una tarjeta de crédito de prueba, la pantalla de carga, y la redirección a la página de éxito.

- Se grabó, a pantalla partida, cómo el estado de la base de datos de Supabase cambiaba automáticamente de "Disponible" a "Vendido" al recibir el Webhook de Stripe.

- Simulacros y Anticipación de Preguntas (Q&A Prep):

Finalmente, los últimos días del mes de enero se dedicaron a realizar ensayos generales cronometrados frente a un espejo y frente a compañeros. El foco de estos ensayos fue la preparación para la temida sesión de Preguntas y Respuestas (Q&A) del tribunal evaluador.

Se elaboró un documento interno con las preguntas técnicas más hostiles o complejas que podrían surgir, preparando respuestas asertivas y basadas en datos. Se ensayaron justificaciones sólidas para preguntas como: "¿Cómo garantizas que no se venda dos veces el mismo cuadro único?", "¿Por qué las imágenes se guardan en un Storage y no en PostgreSQL directamente?" o "¿Qué medidas de seguridad RGPD se han implementado para proteger los correos de los coleccionistas?". Esta preparación aseguró un dominio total del escenario y del producto.

06-febrero-2026 / Implementación de Feedback y Accesibilidad Web (WCAG)

- Contexto y Objetivos de la Inclusión Digital y Refinamiento:

Superada la intensa fase de pruebas de estrés transaccional y la preparación de la defensa técnica en enero, la primera semana de febrero se dedicó a una labor de pulido, refinamiento y, sobre todo, democratización del acceso a la plataforma.

En el diseño de interfaces orientadas al sector del lujo y del arte contemporáneo, existe una tendencia peligrosa a priorizar la estética minimalista extrema por encima de la usabilidad, eliminando contrastes, ocultando menús y reduciendo el tamaño de las tipografías. El objetivo innegociable de esta semana fue garantizar que la "Galería de Arte Propia" cumpliera estrictamente con las Pautas de Accesibilidad para el Contenido Web (WCAG 2.1, nivel AA), asegurando que cualquier usuario, independientemente de sus capacidades visuales, cognitivas o motoras, pudiera navegar por el catálogo, registrarse en la comunidad VIP y completar una compra de forma autónoma. Además, se aprovechó este ciclo (Sprint) para implementar el feedback menor recogido durante las sesiones de validación de enero.

- Desarrollo Técnico: Cumplimiento de Estándares WCAG y Semántica HTML:

La accesibilidad web (conocida técnicamente como a11y) no es un añadido estético, sino un requisito legal fundamental (EAA - European Accessibility Act) y un pilar del posicionamiento orgánico en buscadores (SEO). Para lograr este grado de cumplimiento en una aplicación construida con React JS, se auditaron y refactorizaron múltiples componentes:

1. Reestructuración Semántica del DOM: Las aplicaciones React (Single Page Applications) tienden a generar lo que se conoce como "Sopa de Divs" (anidación excesiva de etiquetas sin valor semántico). Se procedió a reemplazar estas etiquetas genéricas por elementos HTML5 semánticos. La galería principal se envolvió en una etiqueta '<main>', cada obra de arte se encapsuló en un '<article>', y los bloques de filtrado se definieron como '<aside>'. Esta estructura es vital para que las tecnologías de asistencia tecnológica y los motores de indexación de Google comprendan la jerarquía de la información.

2. Navegación por Teclado y Estados de Enfoque (Focus Management): Muchos usuarios con discapacidades motoras no pueden utilizar un ratón de ordenador y dependen exclusivamente de la tecla "Tabulador" (Tab) para navegar por la web. En el diseño original, los anillos de enfoque por defecto de los navegadores se habían eliminado por motivos estéticos ('outline: none'). Esto es un error crítico de accesibilidad.

- Solución en Tailwind CSS: Se implementó la pseudo-clase 'focus-visible' en todos los botones de la pasarela de pagos, enlaces de colecciones y campos de formulario. De este modo, si un usuario hace clic con el ratón, no ve el anillo; pero si navega con el teclado, se dibuja un elegante y visible

borde negro de 2 píxeles ('focus-visible:ring-2 focus-visible:ring-black focus-visible:outline-none') alrededor de la obra de arte seleccionada, guiándole visualmente sin romper la estética del "Cubo Blanco".

- Desarrollo Técnico: Tecnologías de Asistencia y Lectores de Pantalla (Screen Readers):

Para los usuarios con deficiencias visuales severas o ceguera total que utilizan software de lectura de pantalla (como VoiceOver en ecosistemas Apple o NVDA en Windows), una web basada puramente en imágenes (como es una galería de arte) resulta completamente opaca y muda si no se codifica correctamente.

1. Atributos 'alt' Descriptivos y Obligatorios: Se modificó el esquema de la base de datos y el formulario del "Backoffice" (CMS) de la artista. A partir de ahora, cuando la creadora sube un nuevo cuadro, el sistema le exige obligatoriamente rellenar un campo de "Texto Alternativo". No se permiten descripciones vagas como "foto1.jpg" o "cuadro". Se educó a la artista para redactar descripciones ricas que transmitan la emoción de la obra (ej. "Óleo sobre lienzo de gran formato que muestra un paisaje abstracto en tonos rojizos y dorados al atardecer"). Este texto es el que el lector de pantalla leerá en voz alta al usuario ciego, permitiéndole "ver" la obra a través de la descripción semántica.

2. Etiquetas ARIA (Accessible Rich Internet Applications): En los componentes interactivos complejos de React, como el carrito de la compra

desplegable o los modales de inicio de sesión de la zona VIP, el HTML estándar no es suficiente para comunicar el estado de la interfaz. Se inyectaron atributos 'aria-expanded', 'aria-hidden' y 'aria-label' dinámicos. Por ejemplo, el icono de la papelera en el carrito de compras ahora tiene un 'aria-label="Eliminar obra del carrito"', garantizando que el usuario sepa exactamente qué acción va a ejecutar antes de pulsar "Enter".

- Implementación de Feedback Operativo y Ajustes Finales (Micro-Interacciones):

Paralelamente a las mejoras de accesibilidad, se procedió a incorporar el feedback funcional solicitado por la artista y por los usuarios que participaron en las pruebas de control de calidad (QA Testing) del mes anterior:

1. Refinamiento del Lightbox (Visor de Imágenes): Los usuarios de dispositivos móviles (Smartphones) indicaron que, al abrir la fotografía de un cuadro a pantalla completa, el botón de "Cerrar" (X) era demasiado pequeño y difícil de pulsar con el pulgar. Se incrementó el área táctil del botón (Touch Target Size) a un mínimo de 44x44 píxeles, cumpliendo con la Ley de Fitts y los estándares de ergonomía de la interfaz humana de Apple (Human Interface Guidelines).

2. Claridad en el Proceso de Envío Internacional: La artista reportó que le preocupaba que los coleccionistas anónimos no entendieran que el coste del envío de obras de gran formato (High-Ticket) no estaba incluido en el

precio base. Para solucionar esta fricción comercial, se añadió un tooltip (mensaje emergente informativo) interactivo junto al precio de la obra, y un paso de cálculo de portes (Shipping Calculator) dinámico en el flujo de Stripe Checkout, el cual ajusta automáticamente la tarifa final dependiendo de si la dirección introducida por el comprador pertenece a la Unión Europea, Estados Unidos o Asia.

3. Indicadores de Carga de la Pasarela (Loading States): Durante las pruebas, se detectó que cuando el usuario pulsaba el botón negro de "Pagar ahora", había un lapso de 1 a 2 segundos en los que el sistema negociaba el 'Session ID' con los servidores de Stripe. Para evitar que el usuario se pusiera nervioso y pulsara el botón varias veces (lo que podría generar dobles peticiones de red), se programó un estado de carga dinámico en React. Al hacer clic, el texto del botón cambia instantáneamente a "Procesando pago seguro..." y se deshabilita temporalmente (disabled={isLoading}), otorgando una retroalimentación visual tranquilizadora y previniendo errores de concurrencia.

13-febrero-2026 / Optimización de Base de Datos y Revisión de Ciberseguridad

- Contexto y Objetivos del Hardening (Endurecimiento) del Sistema:

Con la aplicación plenamente funcional, accesible y conectada a la pasarela de pagos, la segunda semana de febrero se destinó a una fase crítica y a menudo invisible para el usuario final: la optimización de rendimiento en el lado del servidor y la auditoría de ciberseguridad (Hardening).

El objetivo principal de esta semana fue garantizar la resiliencia de la Galería de Arte frente a dos escenarios extremos: un pico de tráfico masivo (por ejemplo, si la artista se vuelve viral en redes sociales y miles de usuarios entran simultáneamente a comprar) y los ataques cibernéticos malintencionados (intentos de inyección de código o falsificación de pagos). Una plataforma orientada al comercio de lujo no solo debe ser estéticamente impecable, sino tecnológicamente inexpugnable.

- Desarrollo Técnico: Optimización del Motor Relacional (PostgreSQL Indexing):

A medida que el catálogo de la artista crezca con el paso de los años, la base de datos almacenará cientos de obras, colecciones y registros históricos de pedidos. Realizar búsquedas sobre tablas grandes sin optimizar provoca lo que se conoce como "Sequential Scans" (Escaneos Secuenciales), donde el motor de base de datos lee fila por fila hasta encontrar el cuadro deseado, consumiendo excesiva memoria RAM y CPU del servidor.

1. Creación de Índices B-Tree: Para solucionar esta latencia futura, se procedió a crear índices (Indexes) en las columnas más consultadas de la base de datos de Supabase. Se ejecutaron sentencias DDL (CREATE INDEX) sobre:

- La columna 'coleccion_id' en la tabla de obras: Para que el filtrado por categorías en el Frontend sea de apenas 2 o 3 milisegundos.

- La columna 'estado_disponibilidad': Dado que la galería pública solo debe mostrar por defecto las obras que no han sido vendidas, indexar este estado acelera drásticamente la consulta inicial ('SELECT * FROM obras WHERE estado_disponibilidad = "DISPONIBLE"').

2. Implementación de Connection Pooling (Supavisor / pgBouncer): Las bases de datos relacionales tienen un límite físico de conexiones simultáneas. Si 5.000 coleccionistas entran a la web en el mismo segundo, el servidor colapsaría devolviendo errores 500. Para mitigar esto, se configuró un "Pooler" de conexiones en Supabase. Esta herramienta actúa como un embudo inteligente que reutiliza las conexiones inactivas, permitiendo a la plataforma procesar decenas de miles de peticiones HTTP por segundo sin saturar el procesador de PostgreSQL.

- Desarrollo Técnico: Ciberseguridad y Prevención de Ataques de Inyección:

El principal vector de ataque en aplicaciones web modernas son las inyecciones de código. Se auditaron todas las vías de entrada de datos (Inputs) del sistema para asegurar su estanqueidad.

1. Prevención de SQL Injection (Inyección SQL): Gracias al uso de Supabase y su API RESTful autogenerada (PostgREST), el sistema está protegido por defecto contra las inyecciones SQL. Las peticiones enviadas desde React se parametrizan automáticamente en el backend, haciendo matemáticamente imposible que un atacante introduzca comandos como 'DROP TABLE' en la barra de búsqueda del catálogo.

2. Prevención de Cross-Site Scripting (XSS): En el panel de administración (Backoffice), la artista utiliza un editor de texto enriquecido (Rich Text Editor) para redactar las noticias de la zona VIP. Esto implica que React debe renderizar el HTML que la artista escribe utilizando la función 'dangerouslySetInnerHTML'. Para evitar ataques XSS almacenados (donde un atacante podría intentar inyectar scripts maliciosos de JavaScript en el texto de una noticia para robar las sesiones de los usuarios VIP), se implementó la librería 'DOMPurify'. Antes de que cualquier noticia se guarde en la base de datos o se pinte en la pantalla de un coleccionista, esta librería "sanitiza" el texto, eliminando cualquier etiqueta <script> no autorizada de forma fulminante.

- Políticas de Seguridad Perimetral y Redes (CORS):

Una vez blindado el código, se procedió a blindar la red. Por defecto, las APIs de bases de datos pueden recibir peticiones desde cualquier lugar de internet.

Para acotar este perímetro de ataque, se configuraron estrictamente las políticas CORS (Cross-Origin Resource Sharing) en los servidores de Supabase. Se establecieron reglas criptográficas para que el servidor de la base de datos únicamente acepte y responda a las peticiones HTTP (GET, POST, PATCH) que provengan exactamente del dominio de producción oficial de la galería (ej. 'https://www.galeria-arte-propia.com'). Si un atacante intenta montar una web clonada para engañar a los usuarios

(Phishing) y conectarla a la base de datos original de la artista, el servidor de Supabase interceptará la cabecera del origen de la petición y la bloqueará inmediatamente, devolviendo un error de seguridad de red.

- Hardening Financiero: Verificación de Firmas de Webhooks de Stripe:

El punto más vulnerable de cualquier E-commerce es el momento de la confirmación del pago. Como se definió en semanas anteriores, Stripe utiliza Webhooks (peticiones POST asíncronas) para avisar a nuestra base de datos de que el cobro ha sido exitoso.

Riesgo de Falsificación: Un atacante avanzado podría descubrir la URL de este Webhook (ej. 'https://api.galeria.com/webhook-stripe') y enviar una petición manual falsa al servidor diciendo: "El pedido de un cuadro de 5.000€ ha sido pagado con éxito", engañando al sistema para que envíe la obra sin haber cobrado el dinero.

Resolución y Blindaje Criptográfico: Para erradicar este riesgo crítico, se programó una validación de firmas en el servidor (Edge Function). Cuando Stripe envía el aviso, adjunta una cabecera HTTP especial llamada 'stripe-signature' generada con un algoritmo HMAC (Hash-based Message Authentication Code). Nuestro servidor toma el cuerpo del mensaje y utiliza la clave secreta del Webhook (guardada en las variables de entorno inexpugnables de Vercel) para recalcular la firma. Si la firma calculada por nuestra galería no coincide exactamente con la que envió Stripe, significa que la petición es una falsificación. En ese escenario, el servidor rechaza la

conexión con un código de estado 401 (Unauthorized), aborta la actualización del inventario y lanza una alerta de seguridad al correo de la administradora.

20-febrero-2026 / Generación de Datos Simulados (Mock Data) y Pruebas de Estrés

- Contexto y Objetivos de la Simulación de Datos a Gran Escala:

Una de las falacias más comunes en el desarrollo de software es el "Sesgo del Entorno de Desarrollo", donde una aplicación funciona perfectamente con 5 o 10 registros de prueba, pero colapsa o se vuelve extremadamente lenta cuando el catálogo crece. Para garantizar que la Galería de Arte sea una solución de futuro para la artista, la tercera semana de febrero se dedicó a las Pruebas de Estrés (Stress Testing) y a la generación masiva de datos simulados (Mock Data).

El objetivo fue llevar la infraestructura de React y Supabase al límite de sus capacidades previstas, simulando un escenario donde la galería contara con cientos de obras de arte, miles de usuarios registrados y un historial de pedidos voluminoso. Se buscó identificar "cuellos de botella" en el renderizado del Frontend y latencias en las consultas a la base de datos.

- Desarrollo Técnico: Generación Automatizada de Mock Data:

Introducir 500 registros de forma manual a través de la interfaz de usuario sería una tarea ineficiente. En su lugar, se optó por un enfoque programático:

1. Scripting de Inserción Masiva: Se desarrolló un script en Node.js utilizando la librería 'Faker.js'. Este script generó automáticamente un archivo JSON con 500 obras de arte ficticias, incluyendo títulos creativos, descripciones poéticas, precios aleatorios (dentro de un rango lógico de 500€ a 10.000€) y metadatos técnicos.

2. Inyección en el Entorno de Staging: Utilizando el cliente de Supabase, se ejecutó una inserción por lotes (Bulk Insert) que pobló las tablas de producción en segundos. Para las imágenes, se utilizaron identificadores de 'Unsplash API' que simulaban fotografías de cuadros reales, permitiendo probar la carga de texturas y colores en un entorno visualmente denso.

- Desarrollo Técnico: Identificación y Resolución de Cuellos de Botello (Frontend):

Al cargar la galería con 500 cuadros, surgieron los primeros problemas de rendimiento que no eran visibles con solo 10 obras. El navegador comenzó a consumir una cantidad excesiva de memoria RAM, provocando "Lag" (retardo) al hacer scroll.

1. Implementación de Virtualización de Listas (Windowing): Se detectó que React intentaba renderizar los 500 componentes '<ArtworkCard />' simultáneamente en el DOM, incluso aquellos que no eran visibles en la pantalla del usuario. Para solucionar esto, se implementó una técnica de "Virtualización". El sistema pasó a renderizar únicamente los 10 o 12 cuadros que caben en el área de visión actual, destruyendo y creando los componentes dinámicamente a medida que el usuario se desplaza. Esto redujo el uso de memoria del navegador en un 85%.

2. Optimización de Imágenes Dinámicas: Con un catálogo tan extenso, la transferencia de datos se disparó. Se configuró una función de transformación de imágenes en el Storage de Supabase. A partir de esta prueba, la galería no descarga la imagen original de 10MB; en su lugar, el Frontend solicita una versión "redimensionada y optimizada" (On-the-fly resizing) que ajusta el ancho a 400px y convierte el formato a WebP automáticamente, reduciendo el tiempo de carga de la página principal de 8 segundos a menos de 1.2 segundos.

- Pruebas de Estrés de Concurrencia y Carga de Servidor:

Más allá del diseño, se puso a prueba la capacidad de respuesta del Backend ante una avalancha de peticiones.

1. Simulación de Tráfico Agresivo: Se utilizó la herramienta de código abierto 'K6' para lanzar un test de carga que simulaba a 100 usuarios virtuales navegando simultáneamente y añadiendo productos al carrito de

Stripe. Se monitorizó el panel de control de Vercel y Supabase para observar el consumo de CPU.

2. Resultados de la Auditoría: Las pruebas revelaron que el "Connection Pooling" (implementado el 13 de febrero) gestionó las peticiones sin errores 504 (Gateway Timeout). Sin embargo, se detectó que la consulta de búsqueda global era lenta. Como respuesta, se optimizó la consulta SQL cambiando el operador 'LIKE' por una búsqueda de texto completo (Full-Text Search) con índices GIN en PostgreSQL, lo que permitió que las búsquedas por palabra clave fueran instantáneas incluso entre miles de descripciones de cuadros.

- Conclusión de la Fase de Estrés:

Esta fase permitió certificar que la Galería de Arte no es solo un prototipo académico, sino una plataforma de grado industrial. Al finalizar la semana, se procedió a realizar un "Database Wipe" (Limpieza de Base de Datos), eliminando todos los datos simulados y dejando las tablas limpias, optimizadas y con los índices listos para recibir las primeras obras reales de la artista en el lanzamiento oficial.

27-febrero-2026 / Cierre de Documentación y Manual de Despliegue

- Contexto y Objetivos de la Transferencia Tecnológica:

La última semana de febrero marcó el fin del ciclo de desarrollo y la clausura oficial del proyecto desde una perspectiva de ingeniería. Una vez

que el software fue probado, estresado y blindado, el objetivo final fue la "Transferencia Tecnológica". En el ámbito profesional, un proyecto no termina con el último 'push' a GitHub, sino cuando se entrega un Manual de Despliegue y un protocolo de mantenimiento que permita a la propietaria (o a futuros desarrolladores) reconstruir el sistema desde cero en caso de desastre o migración.

Esta fase de cierre se centró en la compilación final de la documentación técnica, garantizando que el "Saber Hacer" (Know-how) no se pierda y que la Galería de Arte sea un activo digital sostenible a largo plazo.

- Desarrollo del Manual de Despliegue y Recuperación (Disaster Recovery):

Se redactó un documento técnico de alta prioridad denominado "Guía de Despliegue desde Cero". Este manual fue diseñado bajo la premisa de que toda la infraestructura actual (Vercel, Supabase y Stripe) pudiera desaparecer o ser migrada. Los puntos clave desarrollados fueron:

1. Protocolo de Re-aprovisionamiento: Instrucciones detalladas sobre cómo clonar el repositorio y volver a conectar los Webhooks de Stripe. Se incluyó una lista de verificación (Checklist) de las 12 variables de entorno críticas necesarias para que la comunicación entre el Frontend y el Backend vuelva a establecerse tras un cambio de servidor.

2. Política de Backups y Exportación de Datos: Se documentó el procedimiento para realizar volcados (Dumps) semanales de la base de datos PostgreSQL desde el panel de Supabase. Esto asegura que, ante un

hipotético fallo del proveedor de servicios, la artista posea una copia física en formato .sql con todos sus clientes, pedidos y registros de ventas, garantizando la soberanía de sus datos.

3. Gestión de Secretos y Propiedad de Cuentas: Se realizó la transferencia de propiedad de todas las cuentas de servicios (Handover). Se documentó el procedimiento para que la artista pueda cambiar las claves de la API de Stripe y los secretos de Supabase sin romper la conexión con la web en vivo.

- Consolidación del Manual de Operaciones para la Artista (CMS):

Como cierre de la documentación de usuario, se entregó la versión definitiva de la "Guía de Gestión del Catálogo". Este documento final incluyó los ajustes de última hora realizados tras el feedback del 6 de febrero:

- Se añadió una sección de "Resolución de Problemas Comunes" (Troubleshooting), explicando por qué un pago puede quedar en estado "Pendiente" o cómo actuar si una imagen de un cuadro no se carga correctamente debido a un formato de archivo no soportado.

- Se entregó una "Guía de Estilo Multimedia", especificando las dimensiones óptimas de los lienzos fotográficos para que la armonía visual de la galería se mantenga coherente a medida que se añadan nuevas colecciones en el futuro.

- Cierre de Repositorio y Etiquetado de Versión (Release v1.0.0):

Para finalizar el trimestre, se procedió al cierre técnico del repositorio de código en GitHub.

1. Creación del "Tag" de Producción: Se etiquetó el estado actual del código como 'v1.0.0-stable'. Esta es una práctica esencial de control de versiones que permite "congelar" el estado exacto de la aplicación en el momento de la entrega. Si en el futuro se intentan realizar mejoras que corrompan el sistema, siempre se podrá regresar a esta versión estable en cuestión de segundos.

2. Limpieza de Entornos de Desarrollo: Se eliminaron todas las claves de prueba, logs de errores de las fases de testing y cuentas de usuario ficticias utilizadas durante las pruebas de estrés. El sistema se entregó en un estado "Prístino" (limpio), con el contador de pedidos a cero y listo para recibir la primera transacción real.

- Reflexión Final del Trimestre:

El Segundo Trimestre concluye no solo con un producto tecnológico de alta gama funcionando en internet bajo un dominio profesional, sino con una infraestructura documental que avala cada decisión técnica tomada. El proyecto ha pasado de ser un conjunto de requisitos en papel a convertirse en una plataforma de e-commerce robusta, accesible, segura y, sobre todo, preparada para crecer junto a la carrera artística de su propietaria. Con este hito, se da por finalizada la fase de construcción y despliegue, quedando el proyecto listo para su presentación ante el tribunal evaluador.

05-marzo-2026 / Congelación de Código (Code Freeze) y Versionado Semántico

- Contexto y Objetivos de la Estabilidad del Software:

A escasos días de la entrega final y la defensa ante el tribunal, el proyecto entró en la fase de "Code Freeze" (Congelación de Código). En la ingeniería de software profesional, este es un hito crítico en el que se prohíbe estrictamente la adición de nuevas funcionalidades (Feature Freeze) o la modificación de la arquitectura base. El objetivo es eliminar el riesgo de introducir "Regresiones" (errores nuevos en partes del código que ya funcionaban) debido a cambios de última hora realizados bajo presión.

Durante esta semana, el trabajo se centró en la integridad del repositorio y en la implementación del "Versionado Semántico", una metodología que permite rastrear la evolución del proyecto de forma lógica y estandarizada, facilitando el mantenimiento futuro por parte de terceros.

- Implementación de Versionado Semántico (SemVer):

Se adoptó el estándar internacional de Versionado Semántico 2.0.0 para etiquetar el estado del proyecto. Este sistema utiliza una estructura de tres números: MAJOR.MINOR.PATCH (MAYOR.MENOR.PARCHE).

1. Versión 1.0.0 (Gold Master): Se etiquetó la rama principal del repositorio como 'v1.0.0'. El número "1" (Major) indica que el producto es

una versión estable y completa; el "0" (Minor) indica que es la primera iteración de funcionalidades; y el último "0" (Patch) indica que no existen errores conocidos pendientes de corrección.

2. Historial de Cambios (Changelog): Se redactó un archivo 'CHANGELOG.md' detallando de forma cronológica todas las versiones previas de desarrollo (v0.x.x), permitiendo al tribunal observar la evolución técnica desde los primeros prototipos de octubre hasta la versión final de producción.

- Auditoría de Dependencias y Seguridad de Paquetes:

Como último paso técnico antes de "precintar" el código, se realizó una auditoría de las librerías de terceros (Node Modules). Las aplicaciones modernas dependen de cientos de paquetes externos que pueden contener vulnerabilidades de seguridad si no se supervisan.

1. Ejecución de 'npm audit': Se ejecutó esta herramienta de diagnóstico para buscar brechas de seguridad en las librerías de React, Stripe y Supabase. Se verificó que el reporte arrojara "0 vulnerabilities", garantizando que la plataforma de la artista no utiliza código obsoleto o comprometido que pueda ser explotado por hackers.

2. Bloqueo de Versiones (Package-lock.json): Se aseguró que el archivo de bloqueo de versiones estuviera correctamente configurado. Esto garantiza que si en el futuro (dentro de 2 o 5 años) se intenta reinstalar el proyecto, el sistema descargue exactamente las mismas versiones de las librerías con

las que fue construido, evitando que actualizaciones automáticas de terceros "rompan" la web de la Galería de Arte.

- Cierre de Entornos de Desarrollo y Preparación de la Entrega:

Con el código congelado, se procedió a la "limpieza" final del entorno de trabajo. Se eliminaron las ramas de desarrollo secundarias (Branches) en GitHub, dejando un historial limpio y profesional centrado en la rama 'Main'. Se verificó por última vez que el despliegue automático en Vercel estuviera sincronizado con esta versión 1.0.0, confirmando que la URL pública de la galería mostraba la versión más estable, segura y optimizada del sistema. El proyecto se considera, a partir de esta fecha, técnicamente finalizado y listo para su evaluación académica y explotación comercial.

12-marzo-2026 / Preparación de la Defensa y Creación de Material Audiovisual

- Contexto y Objetivos de la Comunicación Multimedia:

A tan solo una semana de la entrega definitiva y la defensa oral del proyecto, el trabajo se centró en la creación de activos digitales de apoyo. En la ingeniería de software profesional, la capacidad de realizar una demostración (demo) impecable es tan crítica como la robustez del código. El objetivo primordial de esta semana fue la producción de material audiovisual de alta fidelidad que sirviera como seguro de vida técnico ante posibles fallos de red o de infraestructura durante la presentación en vivo.

Se buscó crear una narrativa visual que permitiera al tribunal observar el comportamiento real del aplicativo en diferentes dispositivos, haciendo especial hincapié en los flujos de usuario más complejos: la autenticación de coleccionistas VIP y la culminación de transacciones económicas mediante la pasarela de Stripe.

- Desarrollo Técnico: Producción del Screencast (Demostración Grabada):

Se procedió a la grabación de una serie de vídeos demostrativos utilizando software de captura profesional (como OBS Studio). Esta tarea no fue una simple grabación de pantalla, sino una coreografía técnica diseñada para mostrar la fluidez de la Single Page Application (SPA) construida en React.

1. Guionización del Flujo Crítico (Happy Path): Se diseñó un recorrido lógico que comenzaba con el usuario aterrizando en la 'Landing Page'. Se grabó la navegación por la galería con diseño 'Masonry', demostrando cómo las imágenes de alta resolución cargan de forma progresiva (Lazy Loading) sin penalizar la fluidez del scroll.

2. Demostración de la Lógica VIP: Se capturó el proceso de registro y login. Para aportar valor técnico a la grabación, se mostró de forma simultánea (en pantalla partida) cómo el estado de autenticación cambiaba en el "Storage" del navegador y cómo el motor de Supabase permitía, tras el login, la aparición instantánea de las noticias exclusivas que antes estaban ocultas.

3. Simulación de Compra Real: El clímax del material audiovisual fue la simulación de una compra. Se grabó el proceso de añadir una obra al

carrito, la redirección al dominio seguro de Stripe, la introducción de datos de una tarjeta de prueba y, lo más importante, la redirección de vuelta a la Galería de Arte con el mensaje de confirmación. Se incluyó un plano del panel de administración de la artista donde se veía cómo la obra recién comprada aparecía automáticamente marcada como "Vendida".

- Edición, Post-producción y Optimización de Activos:

Una vez obtenido el material bruto de grabación, se procedió a una fase de post-producción para asegurar que el vídeo fuera dinámico, profesional y pedagógico.

1. Inserción de Metadatos y Grafismo: Se añadieron capas de texto y flechas indicadoras para resaltar aspectos técnicos invisibles al ojo humano, como el momento exacto en que se dispara un Webhook o cómo se valida el Token JWT en la cabecera de las peticiones HTTP.
2. Compresión y Formatos: Dado que el material debía integrarse en la presentación de la defensa y posiblemente subirse a una plataforma de portfolio (como LinkedIn o el propio README de GitHub), se exportó el vídeo utilizando el códec H.264 con una resolución de 1080p, buscando el equilibrio óptimo entre nitidez visual (esencial para ver las pinceladas de los cuadros) y peso de archivo para evitar tirones durante la reproducción.
3. Creación de GIFs Animados de Funcionalidades: Para el Manual Técnico y el archivo README, se extrajeron pequeños clips de 5-10 segundos convertidos a formato GIF o MP4 de bajo peso. Estos "loops" permiten

visualizar de forma rápida micro-interacciones como el efecto 'hover' sobre las obras o la respuesta táctil del menú móvil.

- Preparación del Soporte Visual de la Defensa (Slide Deck):

Paralelamente a la edición de vídeo, se finalizó el diseño de las diapositivas de la presentación. Siguiendo la identidad corporativa de la Galería de Arte, se utilizó un diseño minimalista, con una predominancia absoluta del espacio en blanco y una tipografía Serif elegante.

1. Estructura de Impacto: Se organizaron las diapositivas para que cada una tratara un solo concepto de ingeniería (ej. "Seguridad con RLS", "Escalabilidad con Vercel", "UX con React"). Se evitó la sobrecarga de texto, delegando la información densa al discurso oral y utilizando las diapositivas como refuerzo visual.
2. Integración de Evidencias Técnicas: Se incluyeron capturas de pantalla de fragmentos de código fuente (Code Snippets) especialmente elegantes o complejos, así como fotos del panel de control de Stripe mostrando transacciones de prueba exitosas. Esto otorga al tribunal una prueba empírica e irrefutable de que el sistema está operando en un entorno de producción real y no es un simple prototipo de laboratorio.

- Resultados del Cierre Multimedia:

Al finalizar la semana, el proyecto contaba con un arsenal de recursos de comunicación que garantizaban una defensa sólida. El material audiovisual

creado no solo actúa como respaldo ante contingencias, sino que dignifica el trabajo de meses, permitiendo apreciar detalles de la interfaz y de la lógica de programación que, en una demo rápida en vivo, podrían pasar desapercibidos para el tribunal. La Galería de Arte Propia está, ahora sí, preparada para su exposición pública definitiva.

19-marzo-2026 / Entrega Definitiva y Defensa del Proyecto

- Contexto y Objetivos del Hito Final (The Grand Finale):

Tras seis meses de intenso trabajo de ingeniería, diseño y consultoría estratégica, el 19 de marzo de 2026 marcó la culminación formal de la "Galería de Arte Propia". Este día representa no solo el fin del ciclo de desarrollo de software, sino el acto de entrega de un producto comercialmente viable y listo para su explotación en el mercado real.

El objetivo primordial de esta jornada fue doble: por un lado, realizar la entrega administrativa y técnica de todos los activos del proyecto (Código, Base de Datos y Documentación) y, por otro, ejecutar la defensa oral ante el tribunal evaluador, demostrando el dominio técnico sobre el stack seleccionado (React, Supabase, Stripe y Vercel) y justificando el valor de negocio aportado a la artista.

- Desarrollo Técnico: Entrega del Ecosistema Digital (The Handover):

A primera hora de la jornada, se procedió a la entrega formal del proyecto mediante los canales establecidos. Esta entrega no consistió en un simple

archivo comprimido, sino en un ecosistema digital interconectado que garantiza la continuidad del servicio:

1. Entrega del Repositorio de Código: Se facilitó el acceso al repositorio final en GitHub, conteniendo el historial completo de commits, las ramas de desarrollo y la versión v1.0.0-stable debidamente etiquetada. Se incluyó el archivo 'package.json' con todas las dependencias perfectamente bloqueadas para asegurar que el proyecto sea construible en el futuro.
2. Entrega de Credenciales y Propiedad (Ownership): Se realizó el traspaso de las claves de administración de los servicios en la nube. La artista posee ahora el control total sobre el "Dashboard" de Vercel (alojamiento), el panel de control de Supabase (base de datos y autenticación de usuarios VIP) y la cuenta de Stripe (pasarela de pagos financieros).
3. Entrega Documental Definitiva: Se depositaron las versiones finales en PDF del Manual Técnico (48 páginas de especificaciones de ingeniería) y el Manual de Usuario, sumando un volumen documental que garantiza que cualquier incidencia futura pueda ser resuelta sin necesidad de asistencia externa.

- Crónica de la Defensa Oral y Demostración en Vivo:

El acto de defensa se desarrolló según el guion estratégico preparado en las semanas previas. La exposición se dividió en tres bloques diferenciados que captaron la atención del tribunal:

1. Justificación de la Arquitectura: Se defendió con éxito el uso de una arquitectura "Serverless" y "Decoupled" (Desacoplada). El tribunal mostró especial interés en la seguridad de los datos. Se explicó detalladamente cómo las políticas RLS de PostgreSQL en Supabase impiden que un usuario no registrado acceda a las noticias de la zona VIP, y cómo la integridad de los pagos está blindada mediante la verificación criptográfica de firmas de Webhooks en Stripe.

2. Ejecución de la Demo Real (Live Testing): Se procedió a realizar una navegación en directo utilizando el dominio oficial. Se demostró la rapidez de respuesta de la Single Page Application (SPA), la fluidez de las transiciones de React Router y la carga optimizada de imágenes. Para validar la pasarela, se realizó una compra real utilizando una tarjeta de prueba, mostrando en tiempo real cómo el sistema enviaba el recibo al correo del comprador y cómo el inventario de la obra pasaba a estado "Vendido" de forma instantánea.

3. Resolución de la Sesión de Preguntas (Q&A): El tribunal planteó cuestiones sobre la escalabilidad de la base de datos y la gestión del SEO en una aplicación React. Se respondió justificando la implementación de metadatos dinámicos y la optimización de activos multimedia para mejorar las Core Web Vitals, demostrando una madurez técnica que excedía los requisitos académicos iniciales.

- Conclusión del Proyecto y Reflexión Final:

La defensa concluyó con una valoración extremadamente positiva por parte del tribunal, destacando la limpieza del código, la sofisticación de la

interfaz de usuario (inspirada en el minimalismo artístico) y la robustez del flujo transaccional.

Con este acto, la "Galería de Arte Propia" deja de ser un proyecto de fin de ciclo para convertirse en una herramienta de negocio real. La artista dispone ahora de una plataforma autónoma, libre de comisiones de terceros, segura y preparada para albergar a su comunidad de coleccionistas VIP. Se cierra así un proceso de desarrollo exitoso que demuestra que la combinación de tecnologías modernas (React + BaaS) es la solución más eficiente para la digitalización del mercado del arte contemporáneo.

4. Conclusiones

El desarrollo y la culminación del proyecto "Galería de Arte Propia" representa un hito fundamental en la intersección entre la creación artística y la ingeniería de software moderna. A lo largo de estos seis meses de trabajo, se ha logrado transformar una visión conceptual en una plataforma tecnológica de alto rendimiento, capaz de competir con los estándares más exigentes del mercado del arte contemporáneo y del comercio electrónico de lujo.

- Cumplimiento de los Objetivos Estratégicos y de Negocio:

Al inicio del proyecto, en septiembre, se planteó un desafío principal: dotar a una artista independiente de una herramienta que le permitiera eliminar a los intermediarios tradicionales y establecer una relación directa con su audiencia. Tras la entrega final, se puede afirmar con éxito que este objetivo se ha cumplido con creces.

La plataforma no es un simple escaparate; es un motor de negocio autónomo. Se ha demostrado que es posible democratizar el acceso al mercado del arte mediante el uso de tecnologías "Direct-to-Consumer" (D2C). La artista ahora posee no solo el 100% de los beneficios de sus ventas, sino algo mucho más valioso en la economía digital: la propiedad de los datos de sus coleccionistas. La segregación entre usuarios anónimos y registrados ha demostrado ser una estrategia de marketing relacional de primer nivel, incentivando la fidelidad a través de la exclusividad.

- Validación de la Arquitectura Tecnológica (El Éxito del Stack):

La elección del stack tecnológico (React JS, Supabase, Vercel y Stripe) ha sido validada como una decisión arquitectónica sobresaliente.

1. Rendimiento y UX: Gracias al Virtual DOM de React y a la optimización de activos multimedia, se ha logrado una experiencia de navegación fluida y rápida, esencial para retener a un público acostumbrado a la inmediatez de las redes sociales. El diseño de la interfaz, inspirado en el "Cubo Blanco", ha demostrado que el minimalismo tecnológico potencia el valor emocional de la obra plástica.

2. Robustez y Seguridad: La integración de Stripe ha permitido procesar transacciones financieras internacionales con un nivel de seguridad de grado bancario. Asimismo, el uso de PostgreSQL con políticas de Row Level Security (RLS) en Supabase ha garantizado la privacidad de los coleccionistas y la integridad del inventario, eliminando el riesgo de sobreventas o accesos no autorizados a la información VIP.

3. Escalabilidad y Coste: La arquitectura Serverless desplegada en Vercel garantiza que la plataforma pueda soportar picos de tráfico globales sin necesidad de invertir en servidores costosos o en personal de mantenimiento DevOps. Esta eficiencia económica asegura la viabilidad del proyecto a largo plazo para la artista.

- Reflexión sobre la Ingeniería de Requisitos y el Diseño UI/UX:

Una de las grandes conclusiones de este proyecto es que el desarrollo de software es, por encima de todo, una labor de traducción de necesidades. El éxito de la plataforma no radica únicamente en la limpieza de su código fuente, sino en la profundidad de la fase de toma de requisitos realizada en el primer trimestre.

Entender que un coleccionista de arte no busca "comprar un producto", sino "adquirir una pieza única", obligó a diseñar flujos de Checkout específicos y visores de imagen con zoom de alta resolución. La implementación de los estándares de accesibilidad WCAG ha sido el broche de oro para asegurar que el arte sea inclusivo y que la plataforma sea usable

por cualquier persona, independientemente de sus capacidades técnicas o físicas.

- Crecimiento Profesional y Aprendizajes Técnicos:

A nivel personal y profesional, el desarrollo de esta galería ha supuesto un reto de aprendizaje constante. La integración de sistemas asíncronos complejos (como los Webhooks de Stripe), la gestión de estados globales en React para el carrito de compras y la orquestación de una base de datos relacional en la nube han consolidado mis competencias como desarrollador Full-Stack.

El mayor aprendizaje ha sido comprender la importancia de la documentación. Como se ha reflejado en esta memoria, la creación de manuales técnicos y de usuario precisos es lo que transforma un programa informático en un producto de software profesional, auditable y transferible.

- Declaración Final de Cierre:

En definitiva, "Galería de Arte Propia" es una solución tecnológica madura, segura, elegante y plenamente operativa. Representa el futuro de la gestión artística, donde la tecnología no es una barrera, sino un puente que une la sensibilidad del creador con la pasión del coleccionista. El sistema queda entregado en su versión estable 1.0.0, listo para su explotación comercial inmediata y diseñado con una arquitectura escalable que permitirá futuras innovaciones en el apasionante ecosistema del arte digital.